Cobweb Forests: Kolmogorov–Arnold Networks Without Gradients

Draft — working notes

October 10, 2026

Abstract

Kolmogorov–Arnold Networks (KANs) place learnable one-variable functions on the edges of a network and justify the design with the Kolmogorov–Arnold representation theorem. We ask whether the same theorem can be realised by Cobweb, an incremental concept-formation system that learns without gradients by sorting each example into a hierarchy of probabilistic concepts. A *Cobweb forest* is a network of Cobweb trees in which the predictions or concept labels of some trees are the inputs of others. We prove three things. *Representation:* one Cobweb tree approximates any continuous function, and a forest of $4n^2 + 4n$ two-input trees realises the Kolmogorov–Arnold representation of any continuous function of n variables; one input per tree is not enough. *Learning:* Cobweb never deletes what it has stored, so a learned tree is a consistent regressor whatever hierarchy it builds, and a learned forest converges to the forest in which every tree predicts the expectation of its outcome given its inputs. *Tasks:* without back-propagation a tree can only learn to predict what it will observe, so the forest must give each tree an observable outcome. Predicting the final target works when the parts combine monotonically and fails on interactions such as x_1x_2; categorizing works for every target but pays for every cell it distinguishes; and no tree can tell its role from its own data, so the choice must be verified where trees are composed. Following the KAN paper's experiments, Cobweb forests do not forget in continual learning and their trees can be plotted like KAN edges, while on smooth targets gradient-trained KANs and MLPs are far more accurate. The discrete cores of the statements are proved in Lean 4 and the rest are checked against the implementation.

Contents

1 Introduction **2**

2 Cobweb trees and forests **4**

3 Representation: a Cobweb–Kolmogorov–Arnold theorem **4**

4 Learning without gradients **5**

4.1 One tree 5

4.2 A forest learns conditional expectations 5

4.3 What should each tree predict? 6

5 Experiments **6**

6 A cognitive reading **9**

7 Discussion **10**

A The model in full **10**
A.1 Concepts and trees . . . 10
A.2 Prediction . . . 11
A.3 Cobweb forests . . . 12

B Representation: statements and proofs **12**
B.1 A local error certificate . . . 13
B.2 One tree . . . 13
B.3 Composition: the Cobweb–Kolmogorov–Arnold theorem . . . 14
B.4 Label forests . . . 15
B.5 Interweaving trees . . . 16

C Learning one tree: statements and proofs **16**

D Tasks: statements and proofs **18**
D.1 Tasks as inputs and outcomes . . . 18
D.2 What a learned forest converges to . . . 19
D.3 Which tasks are sufficient . . . 20

E Choosing tasks: statements and proofs **23**
E.1 Tasks cannot be inferred locally . . . 23
E.2 Verification at the point of composition . . . 24
E.3 Task operators . . . 25

F Experiments in full **25**
F.1 One tree . . . 26
F.2 The KAN battery . . . 27
F.3 Cognitive tasks . . . 28
F.4 Continual learning, interpretability and labels . . . 30
F.5 Real data . . . 30

G Verification **32**

1 Introduction

The Kolmogorov–Arnold (KA) representation theorem [1, 13, 15, 22] says that every continuous $f : [0,1]^n \to \mathbb{R}$ can be written as

$$f(x_1, \ldots, x_n) = \sum_{q=0}^{2n} \Phi_q\Big(\sum_{p=1}^{n} \varphi_{q,p}(x_p)\Big), \tag{1}$$

with continuous one-variable functions $\varphi_{q,p}$ (independent of f) and Φ_q. The only operation on several variables is addition. Kolmogorov–Arnold Networks [14] turn (1) into an architecture: learnable splines on the edges, sums at the nodes, trained by back-propagation.

Cobweb [10] is a different kind of learner. It sorts each example down a tree of probabilistic concepts and, at every node, takes whichever of four operators (add to the best child, create a child, merge two children, split a child) most increases *category utility*, the information the concepts carry about the features [4, 18]. It learns one example at a time, in closed form, with no gradient, and it

	MLP	KAN	Cobweb forest
theorem	universal approximation	Kolmogorov–Arnold	Kolmogorov–Arnold (Theorem 2)
unit	weights, fixed activation	learnable spline on an edge	a Cobweb tree (concept hierarchy)
summation	free, at nodes	free, at nodes	a two-input tree
what a unit passes	a number	a number	a prediction or a concept label
learning	back-propagation	back-propagation	sort and restructure; each tree predicts an observable outcome
forgetting	catastrophic	local	none: no stored example is deleted (Theorem 3)
interpretability	low	plot each edge	plot each tree

Table 1: MLPs, KANs and Cobweb forests.

does not forget. Several systems already wire Cobweb trees together: Convolutional Cobweb [17] feeds the concept labels of patch trees to a classification tree, TRELLIS [21] describes composites by concept identifiers from another tree, and a Cobweb language model passes each tree's prediction to the trees above it. We call such a network a *Cobweb forest* and ask the two questions the KAN paper answers for KANs:

Q1. *Representation.* Can a Cobweb forest represent any continuous function, and with how many trees?

Q2. *Learning.* If every tree learns only by Cobweb's own procedure, does the forest learn the function?

Table 1 lines the three architectures up. The answer to Q1 is the same as for KANs: a forest shaped like (1) represents f (Section 3). Q2 has a twist. A KAN learns its inner functions from the error of their sum. A Cobweb tree receives no error; it can only learn to predict something it will *observe*. The values $\Phi_q(\cdot)$ are never observed, so the KA forest cannot be learned as it stands. What a learned forest can do depends on what each tree is asked to predict, its *task*, and Section 4 shows which tasks work.

Contributions.

- One Cobweb tree is a universal approximator with an explicit rate, and a forest of $4n^2 + 4n$ two-input trees realises (1); one input per tree is not enough (Section 3).
- Cobweb never deletes a stored example, so a learned tree is a consistent regressor whatever hierarchy category utility builds, and a learned forest converges to the forest of conditional expectations its tasks define (Section 4).
- Which tasks recover f: predicting the target, predicting another part of the input, or categorizing; and why a tree's task must be chosen where trees are composed (Section 4.3).
- The KAN paper's experiments repeated for Cobweb forests, plus housing, MNIST and text (Section 5); a short cognitive reading (Section 6); and verification by Lean proofs, executable checks and an adversarial review (Appendix G).

2 Cobweb trees and forests

A tree as a function. A Cobweb tree stores examples (x, y). Every concept c keeps the count n_c of the examples below it and their mean $\mu_c = (\mu_{c,x}, \mu_{c,y})$ and variance; the leaves hold the examples themselves. To predict y at a new x, the tree mixes the means of its leaves, weighting each by how many examples it holds and how well it explains x:

$$\hat{y}(x) = \frac{\sum_{c \text{ leaf}} n_c \, \mathcal{N}(x; \mu_{c,x}, h^2) \, \mu_{c,y}}{\sum_{c \text{ leaf}} n_c \, \mathcal{N}(x; \mu_{c,x}, h^2)}, \tag{2}$$

where the variance floor h^2 plays the role of a bandwidth. This is the *count-weighted leaf mixture*. The implementation also mixes over internal concepts and offers four weightings; Appendix A defines all of them, and the results below say which ones they cover.

Forests. A *Cobweb forest* is a directed acyclic graph of Cobweb trees with one output tree. An edge $\sigma \to \tau$ carries one of two *messages* from σ into τ's examples: its *activation*, the prediction (2) of σ's outcome, or its *label*, the identifier of the concept the input sorts to. A tree may also read raw coordinates of x. Every tree learns with Cobweb's own procedure on the examples made of its inputs and its outcome. In the *online* protocol each tree first emits its message for a new observation and then learns it, as Cobweb's predict-then-learn loop does; this keeps a tree's training messages out of sample (Definition A.6).

3 Representation: a Cobweb–Kolmogorov–Arnold theorem

Theorem 1 (One tree is a universal approximator; Theorem B.2)**.** *Let g be continuous on a compact $K \subset \mathbb{R}^d$. For every $\varepsilon > 0$ there is a Cobweb tree whose prediction is within ε of g on all of K. If g is L-Lipschitz, a tree with G leaves achieves error at most $C_d \, L \, \mathrm{diam}(K) \, G^{-1/d} + C_d \|g\|_\infty G^{-1}$.*

Idea. Put one leaf at each point of a fine grid, holding $(\gamma, g(\gamma))$. With a variance floor at the grid spacing, (2) is a Gaussian average of g over the grid points near x, and the root's share of the mixture is of order $1/G$. The same holds for two of the implementation's all-concept weightings, while the other two keep a fixed bias on such a flat tree (Proposition B.3).

Theorem 2 (Cobweb–Kolmogorov–Arnold; Theorem B.6 and Proposition B.7)**.** *For every continuous f on $[0,1]^n$ and every $\varepsilon > 0$ there is a Cobweb forest of $4n^2 + 4n$ trees, each with at most two inputs and passing activations, whose output is within ε of f (Fig. 1). A forest in which every tree has one input computes a function of a single coordinate, so two inputs per tree are necessary.*

Idea. Read (1) as a graph of one- and two-variable maps: $n(2n{+}1)$ inner functions, $(2n{+}1)(n{-}1)$ additions for the inner sums, $2n + 1$ outer functions and $2n$ additions for the outer sum. Replace every map by a tree from Theorem 1. Errors pass through continuous maps, so choosing each tree's accuracy backwards from the output keeps the total error below ε (Lemma B.5).

Three remarks place the theorem. First, as for KANs, the useful statement assumes structure: if f is a composition of maps with at most k inputs each, a forest with G leaves per tree has error of order $G^{-1/k}$, set by the fan-in and not by n, while a single tree pays $G^{-1/n}$ (Remark B.8). Second, labels are universal too: trees that categorize each coordinate and pass their labels to a lookup tree approximate any f, but the lookup tree needs one leaf per cell, K^n in all, because labels carry no metric (Theorem B.9). Third, merging trees under one common root does not compose them: the merged tree predicts a mixture of their predictions (Proposition B.11). Composition is what a forest adds to a tree.

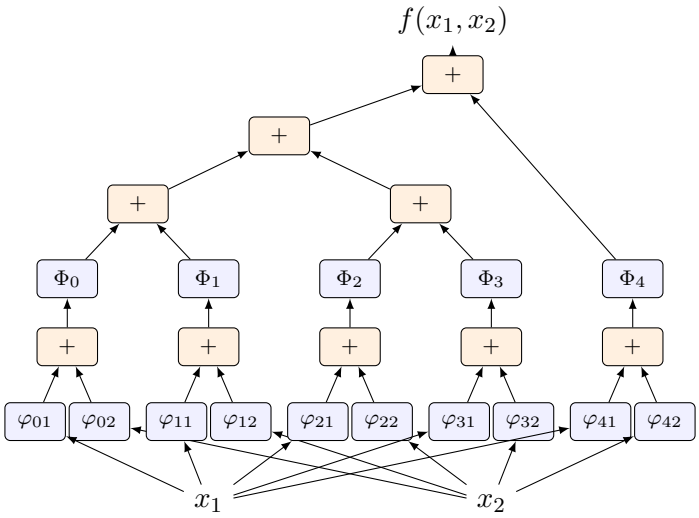


Figure 1: The Cobweb–Kolmogorov–Arnold forest for $n = 2$: every box is a Cobweb tree that passes its activation. Blue trees have one input and realise the functions of (1); orange trees have two inputs and realise the sums. There are $4n^2 + 4n = 24$ trees.

4 Learning without gradients

4.1 One tree

Theorem 3 (Cobweb never forgets, so a learned tree is consistent; Lemma C.1 and Theorem C.2)**.** *None of Cobweb's operators deletes a leaf, and every example ends in exactly one leaf. Hence, whatever hierarchy category utility builds and in whatever order the examples arrive, the count-weighted leaf mixture* (2) *of a learned tree equals the Nadaraya–Watson kernel regression on all examples seen. With exact matching, if $h \to 0$ and $Nh^d \to \infty$, the learned tree's prediction converges to $\mathbb{E}[Y \mid X]$ (almost surely, in L_1) for every distribution of (X, Y) with bounded Y.*

The hierarchy plays no role in consistency; it matters for search under a budget, for the default PATH weighting, and for the labels a tree passes on (Remark C.4). The weights must count examples: a count-blind mixture averages distinct stored values instead of samples and fails when examples repeat (Remark C.3).

4.2 A forest learns conditional expectations

Give every tree τ a *task*: its inputs U_τ (raw coordinates and parents' messages) and an *outcome* V_τ it will observe. By Theorem 3 each tree learns $\mathbb{E}[V_\tau \mid U_\tau]$, so the forest has a natural limit, the *ideal forest*: in topological order, every tree emits $m_\tau = \mathbb{E}[V_\tau \mid U_\tau]$, where U_τ is built from the ideal messages of its parents.

Theorem 4 (A learned forest converges to its ideal forest; Theorem D.5 and Corollary D.6)**.** *Under regularity conditions (bounded outcomes, inputs with densities, variance floors that shrink slowly enough, Assumption D.4), the output of a forest learned by Cobweb converges uniformly, in probability, to the output F of its ideal forest. It therefore learns $f = \mathbb{E}[Y \mid X]$ if and only if $F = f$, i.e. if and only if its tasks are* sufficient.

Theorem 4 separates what Cobweb's learning does from what the tasks determine. Every tree learns exactly the conditional expectation of its outcome given its inputs; whether these compose to f is a property of the tasks alone.

4.3 What should each tree predict?

In the KA forest the tree for Φ_q must hold $\Phi_q(u_q)$, a quantity no one ever observes; only the sum f is observed (Remark D.2). A KAN separates the terms of the sum by back-propagation. A Cobweb tree can only predict an observable outcome, and there are three natural kinds: *funnel* (every tree predicts the target Y, as in the Cobweb language model), *cross-view* (a tree predicts a part of the input it does not see, such as a neighbouring pixel), and *categorization* (no outcome; the tree passes labels, as in Convolutional Cobweb).

Proposition 5 (Which tasks are sufficient; Appendix D)**.**

(a) Funnel tasks work when the parts combine monotonically*: if* $Y = \Phi\big(\sum_j g_j(X_{B_j})\big) + \text{noise}$ *with* Φ *increasing and independent blocks* X_{B_j}*, a forest in which each tree predicts* Y *is sufficient (Proposition D.7).*

(b) Funnel tasks fail on interactions*: for* $Y = X_1X_2$ *with symmetric inputs, a tree that sees only* X_1 *predicts* $\mathbb{E}[Y \mid X_1] = 0$*, so the funnel forest learns nothing (Proposition D.8).*

(c) Categorization works for every target, at a price*: label forests are consistent whenever their cells shrink and stay few compared with the data, conditions that do not depend on* f*; filling* n *dimensions takes a number of cells exponential in* n *(Theorem D.10).*

(d) Cross-view tasks work when the target lives in what the views share*, as in a Gaussian multi-view model (Proposition D.9).*

Can a tree find out by itself whether its task is right? No.

Proposition 6 (A tree cannot tell its role from its own data; Theorem E.2 and Corollary E.3)**.** *Let* X_1, X_2, X_3 *be uniform on* $[-1, 1]$ *and compare* $Y = \operatorname{sign}(X_1X_2)$ *with* $Y = \operatorname{sign}(X_1X_3)$*, observing only* X_1, X_2, Y*. In the first world* X_2 *is essential; in the second it is irrelevant. Yet the pair* (X_2, Y) *has the same distribution in both, so a tree that sees* X_2*, whatever it predicts, receives the same data, and every rule based on its own data that decides whether to keep it errs with probability at least* $1/2$ *in one of the two worlds. The same holds for the parity of any number of signs.*

The information a tree carries can be synergistic: it exists only together with another tree's input. Back-propagation solves this by sending the error of the composition to every module. A Cobweb forest must instead decide at the tree that composes the parts. Choosing among candidate task assignments by the validation error of the output is consistent: if every candidate forest converges and one of them is sufficient, the choice is sufficient with probability tending to one (Theorem E.4). This gives a simple recipe (Procedure E.6): merge the pairs of inputs whose combination the activations lose (those with a positive *synergy deficit*, Definition E.5), compose the groups with two-input trees, and choose each tree's message by validation, always keeping the single merged tree among the candidates.

5 Experiments

We repeat the KAN paper's experiments [14] and add tabular data, images and text. Trees use the implementation's defaults (the PATH weighting over leaves, a budget of 300 leaves, the online protocol), which lie outside the hypotheses of the convergence theorems; the experiments test the

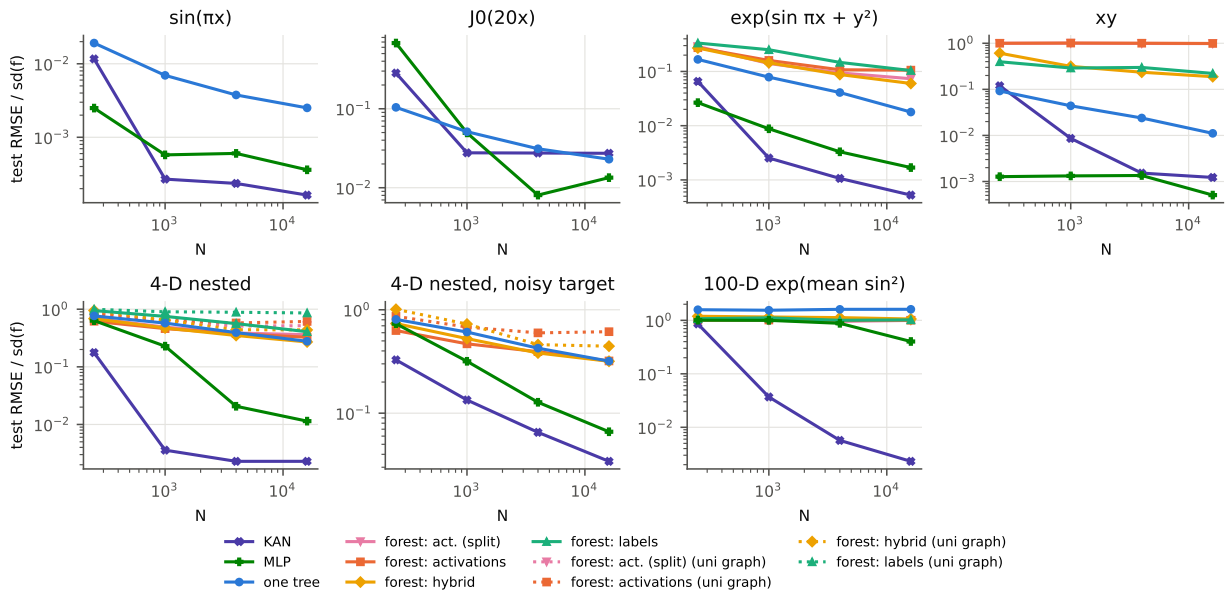


Figure 2: Test RMSE/sd(f) against N on the KAN paper's toy functions. Forests are drawn with their best graph (dotted: the univariate graph of the 4-D target).

theorems' predictions rather than instantiate them. Errors are test RMSE divided by the standard deviation of f, averaged over three seeds unless stated. The full setup and further experiments are in Appendix F. We do not claim that Cobweb forests are competitive regressors: a Cobweb tree is a nonparametric smoother, and on smooth targets the gradient-trained MLP and KAN are far more accurate.

Toy functions (Fig. 2 and Table 2). One tree's error falls with N on every target of dimension up to four, as Theorem 3 predicts; on $\sin(\pi x)$ and $\exp(\sin \pi x_1 + x_2^2)$ at $N = 16000$ it is 7 to 11 times behind the MLP and 16 to 34 times behind the KAN, and on $J_0(20x)$ it beats the KAN, whose grid limits it. On $x_1 x_2$ the funnel forest stays at the error of a constant (0.99 to 1.02), as Proposition 5(b) predicts, while label (0.22) and hybrid (0.19) forests learn. On the 4-D target $\exp\big(\frac{1}{2}(\sin \pi(x_1^2 + x_2^2) + \sin \pi(x_3^2 + x_4^2))\big)$ the funnel forest over the two pairs, which is sufficient by Proposition 5(a), beats the single tree at small N (0.46 against 0.57 at $N = 1000$) and loses at $N = 16000$ (0.33 against 0.28). On the 100-D target no Cobweb design gets below 0.95, while the KAN reaches 0.002: each univariate tree must estimate a weak signal, and its errors compound through the levels (Table 5).

Continual learning (Fig. 3). Theorem 3 says that a tree never deletes what it has learned. On the KAN paper's continual task the tree's error on all data seen so far stays between 0.037 and 0.048 through all five phases; the MLP forgets (its error reaches 10.8) and the KAN with a fine grid forgets partly (0.17; Table 9).

Interpretability (Fig. 4). Like a KAN's edges, a forest's trees can be plotted. On $\exp(\sin \pi x_1 + x_2^2)$ the univariate trees learn $c_1 e^{\sin \pi x_1}$ and $c_2 e^{x_2^2}$, the inner functions of the target up to monotone maps, without gradients.

fn	N	one tree	activations	labels	hybrid	act. (split)	MLP	KAN
sin	250	0.019	–	–	–	–	0.0025	0.012
sin	1000	0.007	–	–	–	–	0.00058	0.00027
sin	4000	0.0038	–	–	–	–	0.0006	0.00024
sin	16000	0.0025	–	–	–	–	0.00036	0.00016
j0	250	0.104	–	–	–	–	0.679	0.283
j0	1000	0.051	–	–	–	–	0.050	0.028
j0	4000	0.031	–	–	–	–	0.0081	0.028
j0	16000	0.023	–	–	–	–	0.013	0.027
expsin	250	0.167	0.274	0.334	0.271	0.289	0.027	0.065
expsin	1000	0.079	0.160	0.252	0.145	0.140	0.0088	0.0025
expsin	4000	0.041	0.107	0.147	0.088	0.094	0.0033	0.0011
expsin	16000	0.018	0.106	0.104	0.060	0.073	0.0017	0.00053
xy	250	0.092	1.002	0.397	0.610	0.997	0.0013	0.119
xy	1000	0.044	1.011	0.291	0.315	1.004	0.0013	0.0086
xy	4000	0.024	1.003	0.299	0.235	0.991	0.0014	0.0015
xy	16000	0.011	0.991	0.222	0.188	0.985	0.00051	0.0012
nested4	250	0.765	0.618	0.946	0.685	0.695	0.633	0.176
nested4	1000	0.574	0.460	0.751	0.472	0.488	0.228	0.0036
nested4	4000	0.393	0.376	0.563	0.350	0.386	0.021	0.0023
nested4	16000	0.277	0.326	0.407	0.270	0.361	0.011	0.0023
nested4 (noisy)	250	0.807	0.630	–	0.735	–	0.733	0.328
nested4 (noisy)	1000	0.609	0.468	–	0.526	–	0.317	0.134
nested4 (noisy)	4000	0.425	0.393	–	0.382	–	0.128	0.065
nested4 (noisy)	16000	0.319	0.320	–	0.318	–	0.066	0.034
exp100	250	1.580	1.007	1.073	1.192	1.027	1.007	0.856
exp100	1000	1.546	1.015	1.114	1.156	1.004	1.000	0.037
exp100	4000	1.606	0.992	1.003	1.125	0.957	0.880	0.0057
exp100	16000	1.609	1.005	1.022	1.060	0.996	0.402	0.0023

Table 2: Toy functions: test RMSE/sd(f), mean of three seeds. "activations" is the funnel forest; for the 4-D target the forests use the pairs graph (superscript u: univariate graph).

Choosing tasks. The sign worlds of Proposition 6 behave as predicted (Table 6): the tree that sees x_2 has the same error and the same flat activation in both worlds, while the composing tree separates them. Choosing among candidate forests by the output's validation error never selects the funnel forest on x_1x_2 or on the sign target, and the selected forest is within one percent of the best candidate in every setting (Table 7). On Feynman equations the recipe of Section 4.3 finds the structure of the formula—in $q(E_f + Bv\sin\theta)$ it groups B, v and θ—and it recovers the two blocks of the 4-D target (Table 8).

Real data. On California housing the funnel forest over natural feature pairs (longitude–latitude, …) beats the single tree at every N (0.276 against 0.303 at $N = 16000$, Table 11); MLP and KAN are better still. On MNIST (Table 3) patch trees that predict the class and pass their predicted distributions reach 94.4%, and the hybrid forest 94.9% at $N = 20000$, level with the KAN (94.5%) and below the MLP (96.8%). Categorizing patch trees reach 86% and cross-view patch trees 37–52%: neighbouring pixels do not carry the class, so Proposition 5(d) does not apply. On next-word prediction from four words (Table 12) generic forests barely improve on one tree; a Cobweb language

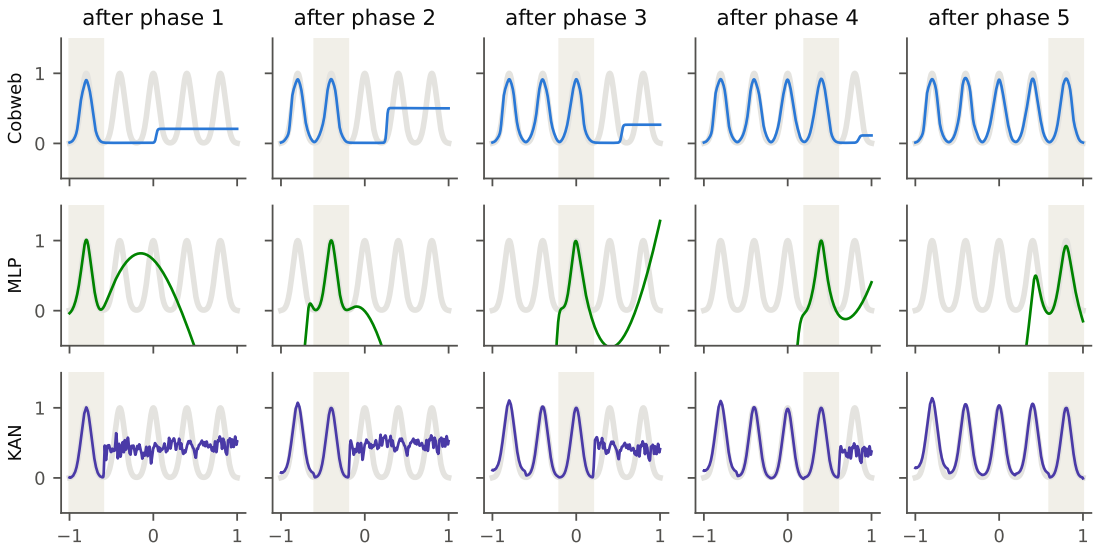


Figure 3: Continual learning on the KAN paper's five-peak task [14, Fig. 3.4]; one region per phase (shaded), target in grey.

N	single	label	cross	act	hybrid	mlp	kan
1000	83.2±1.1	70.7±1.7	36.9±2.1	85.8±0.3	84.2±0.7	88.2±1.1	88.6±0.6
5000	91.4±0.4	81.9±1.2	46.0±0.1	92.4±0.2	92.3±0.5	94.0±0.2	92.2±0.7
20000	94.2±0.3	86.2±1.1	51.9±0.1	94.4±0.5	94.9±0.6	96.8±0.3	94.5±0.7

Table 3: MNIST accuracy (%, three seeds); one tree per 7×7 patch. "label": patch trees categorize (Convolutional Cobweb); "cross": they predict the centre pixels of the neighbouring patches; "act": they predict the class and pass its distribution; "hybrid": "act" plus labels.

model needs its own architecture.

6 A cognitive reading

The three tasks of Section 4.3 are three things a learner can do with what it perceives: *categorize* it, as in Cobweb's original concept formation; *predict an outcome* from it, as in associative learning [20]; or *predict another part* of it, as in predictive-processing accounts of perception [3]. The theory says when each suffices. Categories must grow finer with experience, and so must the trees' acuity, the variance floor h; both are forms of perceptual learning [11]. Outcome prediction compresses a percept to what matters for the outcome, which works when parts combine monotonically and fails on interactions. Perceptual prediction never needs the outcome and works when the outcome lives in what the views share. Two further properties have a cognitive flavour. Cobweb stores and does not overwrite, which is why a tree does not forget (Theorem 3). And Proposition 6 is a precise form of the credit-assignment problem [19]: a module cannot learn its role from what it alone sees; the decision belongs where the parts are combined and checked against an observed outcome.

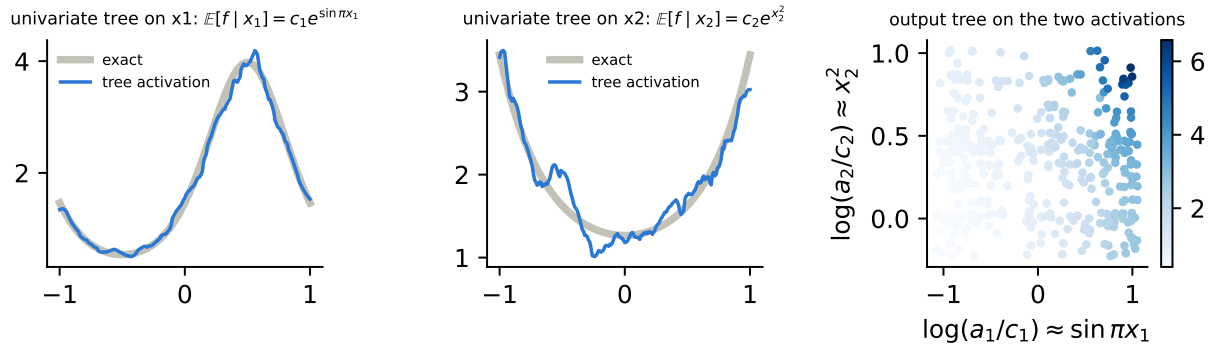


Figure 4: What the trees of a funnel forest learn on $\exp(\sin \pi x_1 + x_2^2)$ ($N = 4000$). Left and centre: each univariate tree's activation against the exact $\mathbb{E}[f \mid x_i]$. Right: the output tree as a function of the two activations.

7 Discussion

Cobweb forest or KAN? Use a KAN or an MLP when the target is smooth, the data are plentiful and offline, and accuracy is what matters: back-propagation fits the inner functions of (1) directly, and no Cobweb design comes close on smooth targets. A Cobweb forest is the choice when learning must be incremental, gradient-free and free of forgetting, when the units should be inspectable concept hierarchies, or when the inputs are symbolic and the messages should be concepts.

Rules of thumb for composing trees.

1. Give every tree an observable outcome, or none (Section 4.3).
2. Pass activations where parts combine monotonically; merge inputs, or pass labels, where they interact (Proposition 5).
3. Choose a tree's task where its message is used, by the output's validation error, and keep the merged tree among the candidates (Proposition 6).
4. Keep forests shallow: two inputs per tree suffice in principle, but each level adds error that the next level reads as noise (Table 5).
5. Predict with counts and train predict-then-learn (Remark C.3 and Definition A.6).

Limitations. The convergence theorems assume the split protocol, full expansion, exact matching and the count-weighted leaf mixture; the experiments use the implementation's defaults. One step of Theorem D.5 relies on a standard uniform law for kernel sums, the label-cell conditions of Theorem D.10 are not derived from category utility, and the task recipe is greedy. The rates are those of nonparametric smoothing.

A The model in full

A.1 Concepts and trees

An *instance* is a pair $z = (a, u)$ of a nominal part a (a table that assigns to each attribute a bag of values with weights) and a continuous part $u \in \mathbb{R}^m$.

Definition A.1 (Concept)**.** A concept c stores its count n_c, the attribute–value counts $n_c(A{=}v)$ of the instances it covers, and, for the continuous block, their mean $\mu_c \in \mathbb{R}^m$ and empirical (per-dimension) variance $s_c^2 \in \mathbb{R}^m_{\geq 0}$. With smoothing parameters $\alpha > 0$ and $h^2 > 0$ (`alpha`, `prior_var`) a concept scores a query with

$$P_c(A{=}v) = \frac{n_c(A{=}v) + \alpha}{n_c(A) + (K_A + 1)\alpha}, \qquad p_c(u_j) = \mathcal{N}\big(u_j;\, \mu_{c,j},\, v_{c,j}\big), \quad v_{c,j} = s_{c,j}^2 + h^2,$$

with attributes conditionally independent given c and K_A the number of values of A seen so far (the extra α reserves mass for an unseen value). When a concept *predicts* a missing attribute it uses $(n_c(A{=}v) + \alpha)/(n_c(A) + K_A\alpha)$, which sums to one over the seen values.

Definition A.2 (Cobweb tree)**.** A Cobweb tree $\mathcal{T}$ is a rooted tree of concepts whose statistics are *additive*: each internal concept's statistics are those of the pooled instances of its children. Its *category utility* at a node with children C is the normalised information gain $\mathrm{PU} = |C|^{-1}\big[H(\text{parent}) - \sum_{c\in C} P(c)H(c)\big]$, where H is the sum of the Dirichlet-smoothed categorical entropies and of the Gaussian differential entropies $\frac{1}{2}\log(2\pi e\, v_{c,j})$. For nominal attributes and $\alpha \to 0$ the bracket is $\sum_A \big[H(A) - H(A \mid C)\big] = \sum_A I(A; C)$, the information the partition carries about each attribute; the Gaussian terms are its analogue for fitted Gaussians. This is the information-theoretic category utility of Corter and Gluck [4], which the rational account reads as the mutual information between features and concepts [18].

Learning (`ifit`) sorts an instance from the root. At a leaf it either increments the leaf (exact match) or performs a *fringe split*: the leaf becomes an internal node with the old leaf and a new leaf for the instance as children. At an internal node it adds the instance and chooses, by category utility, between BEST (descend into the best child), NEW (create a leaf), MERGE (join the two best children under a new node) and SPLIT (replace the best child by its children).

A.2 Prediction

A *query* is an instance in which some attributes and some continuous coordinates are missing; we write O for the observed coordinates (missing continuous coordinates are encoded as `NaN`, missing attributes are absent). The log-likelihood of a query under a concept is $\ell_c(q) = \sum_{A\ \text{obs.}} \log P_c(A{=}q_A) + \sum_{j\in O} \log p_c(q_j)$.

Definition A.3 (Multi-node prediction)**.** Prediction expands concepts best-first from the root, scoring each non-root concept c with one of

$$\begin{aligned}
&\text{LIK}: && s_c = \ell_c(q),\\
&\text{PRIOR}: && s_c = \ell_c(q) + \log(n_c/n_{\text{root}}),\\
&\text{PATH}: && s_c = \pi_c(q) + \ell_c(q), \qquad \pi_c(q) = \textstyle\sum_{c'\in \text{path}(c)\setminus\{\text{root}\}} \log \frac{n_{c'} e^{\ell_{c'}(q)}}{\sum_{b\in \text{sib}(c')} n_b e^{\ell_b(q)}},\\
&\text{POSTERIOR}: && s_c = \pi_c(q),
\end{aligned}$$

where $\text{sib}(c')$ are the children of c''s parent (including c'); under every weighting the root is scored by its likelihood, $s_{\text{root}} = \ell_{\text{root}}(q)$. With $E_M(q)$ the expanded set and $w_c(q) \propto e^{s_c}$ normalised over $E_M(q)$, the prediction of a missing continuous coordinate j and of a missing attribute A is

$$\hat{y}_j(q) = \sum_{c\in E_M(q)} w_c(q)\, \mu_{c,j}, \qquad \hat{P}(A{=}v \mid q) = \sum_{c\in E_M(q)} w_c(q)\, P_c(A{=}v).$$

The budget M bounds the number of expanded concepts. The *leaves-only* variant restricts the mixture to the expanded leaves; its budget counts leaves (expansion stops once M leaves have entered the mixture, after at most $64M$ expansions). *Full expansion* is $M = \infty$.

PATH is the default of the implementation and of `cobweb_torch`; $\pi_c(q)$ is the log posterior of c obtained by multiplying the sibling-normalised posteriors down its path, so $\sum_{c \text{ at depth } k} e^{\pi_c} \leq 1$ for every depth k. The rational account [18] ranks concepts by pointwise mutual information $\log p(x \mid c) - \log p(x)$, i.e. by ℓ_c.

A.3 Cobweb forests

Definition A.4 (Messages)**.** Let $\mathcal{T}$ be a tree whose instances contain *outcome* attributes $V_{\mathcal{T}}$ (possibly none; Definition D.1). For a query q in which $V_{\mathcal{T}}$ is missing:

- the *activation* of $\mathcal{T}$ is its prediction of $V_{\mathcal{T}}$—the vector $\hat{y}_{V_{\mathcal{T}}}(q)$ for a continuous outcome, the predicted distribution $(\hat{P}(V_{\mathcal{T}}{=}v \mid q))_v$ for a nominal one;
- the *label* of $\mathcal{T}$ at depth k is the identifier of the depth-k concept on the root-to-leaf descent that chooses, at each node, the child maximising $\log n_c + \ell_c(q)$ (the leaf if the path is shorter; $k = \infty$ denotes the leaf).

Definition A.5 (Cobweb forest)**.** A Cobweb forest $\mathfrak{F} = (V, E, o)$ is a finite directed acyclic graph whose vertices are Cobweb trees and whose edges $\sigma \to \tau$ carry the activation and/or the label of σ into τ, together with an output tree $o \in V$. A tree τ learns instances made of (a) the raw coordinates x_{S_τ} it is wired to, (b) the activations of its parents as continuous coordinates, (c) the labels of its parents as nominal attributes (one attribute per parent), and (d) its outcome, if any. For an input x the forest computes the messages of every tree in topological order and outputs the activation of o. The *fan-in* of τ is the number of its non-outcome inputs (raw coordinates, activation coordinates, label attributes).

Definition A.6 (Training protocols)**.** Every tree learns only with `ifit`. Two protocols fix the messages a tree is trained on:

- *online*: for each observation i, every tree first emits its message for x_i from what it has learned from observations $1, \ldots, i-1$, then learns observation i (activations are out-of-sample, as at test time; labels are concept identifiers, which can change as the tree restructures, so a child may store labels that its parent no longer emits);
- *split*: the stream is cut into consecutive chunks, one per level of the DAG; a tree at level k learns chunk k, reading the messages of its already-trained (frozen) parents.

Remark A.7 (Why not train stage-wise on the same data)*.* A tree that has learned (x_i, y_i) keeps a leaf for it (Lemma C.1), and when its variance floor is small compared with the spacing of the sample its activation at x_i is close to y_i itself. A parent trained on such in-sample messages learns the relation between y and a leaked copy of y, which it never sees at test time. The online protocol (predict, then learn), already used by the Cobweb language model, and the split protocol avoid this.

B Representation: statements and proofs

Throughout, a tree that *represents* $g : K \to \mathbb{R}$ ($K \subset \mathbb{R}^d$ compact) is a tree over instances $(x, g(x))$ queried with the last coordinate missing, and $\omega_g(r) = \sup\{|g(x) - g(x')| : \|x - x'\| \leq r\}$ is the modulus of continuity.

B.1 A local error certificate

Lemma B.1 (Concept bias)**.** *Let c be a concept of a tree over instances $(x_i, g(x_i))$, with g L-Lipschitz on a convex set containing the instances' x-parts and the queries. Then $|\mu_{c,Y} - g(\mu_{c,X})| \le L\sqrt{\operatorname{tr} S_c}$, where S_c is the empirical covariance of the x-part of c's instances, and consequently, for every query x, every weighting and every budget for which the mixture is not empty,*

$$|\hat{y}(x) - g(x)| \;\le\; L \sum_{c \in E_M(x)} w_c(x) \Big(\|x - \mu_{c,X}\| + \sqrt{\operatorname{tr} S_c} \Big). \tag{3}$$

Proof. $|\mu_{c,Y} - g(\mu_{c,X})| = |\frac{1}{n_c}\sum_i (g(x_i) - g(\mu_{c,X}))| \le \frac{L}{n_c}\sum_i \|x_i - \mu_{c,X}\| \le L(\frac{1}{n_c}\sum_i \|x_i - \mu_{c,X}\|^2)^{1/2}$ by Jensen. For (3), write $\hat{y} - g(x) = \sum_c w_c(\mu_{c,Y} - g(\mu_{c,X})) + \sum_c w_c(g(\mu_{c,X}) - g(x))$ and use $\sum_c w_c = 1$. □

The right side of (3) is computable from the tree: a mixture is accurate when it puts its weight on concepts that are near the query *and* tight.

B.2 One tree

Theorem B.2 (A single tree is a universal approximator)**.** *Let $g \in C(K)$, $K \subset \mathbb{R}^d$ compact, and $\varepsilon > 0$. There is a Cobweb tree $\mathcal{T}$ and a variance floor $h^2 > 0$ such that $\sup_{x \in K} |\hat{y}_{\mathcal{T}}(x) - g(x)| < \varepsilon$ under full expansion with each of the weightings* LIK *and* PATH*, and with the leaves-only variant of any weighting. If g is L-Lipschitz and K lies in a cube of side D, a tree with G leaves achieves*

$$\sup_{x \in K} |\hat{y}_{\mathcal{T}}(x) - g(x)| \;\le\; C_d\, L\, D\, G^{-1/d} + C_d\, \|g\|_\infty\, G^{-1}. \tag{4}$$

Proof. Extend g continuously to a cube $Q \supseteq K$ of side D (Tietze's extension, or McShane's when g is L-Lipschitz, which keeps the constant L; its sup norm is at most $\|g\|_\infty + L\sqrt{d}\,D$, and the extra term is absorbed in the first term of (4)) and rescale Q to $[0,1]^d$, which multiplies the Lipschitz constant by D. Fix $\beta > 0$, let $\delta = \beta h$ and let Γ be the grid of cell centres of spacing δ, $G = |\Gamma| = \delta^{-d}$. Take the depth-one tree whose leaves are the instances $(\gamma, g(\gamma))$, $\gamma \in \Gamma$, and whose root pools them. Each leaf has $s^2 = 0$, hence variance h^2 in every coordinate; the root has variance at least $v_0 > 0$ (the variance of the grid) in every coordinate. Write $k_\gamma = (2\pi h^2)^{-d/2} e^{-\|x-\gamma\|^2/2h^2}$.

Leaves dominate. Some γ lies within $\sqrt{d}\,\delta/2$ of x, so $\max_\gamma k_\gamma \ge (2\pi h^2)^{-d/2} e^{-d\beta^2/8}$, while $\sum_\gamma k_\gamma \le (2\pi h^2)^{-d/2} C(\beta, d)$ by comparison with a Gaussian sum over a grid of spacing β in units of h. Under LIK the leaves carry unnormalised mass $\sum_\gamma k_\gamma \ge c\, h^{-d}$ and the root at most $(2\pi v_0)^{-d/2}$. Under PATH the leaf γ has $e^{\pi_\gamma} = k_\gamma / \sum_{\gamma'} k_{\gamma'}$ (equal counts), so its mass is $k_\gamma^2 / \sum k$, whose sum is at least $(\max k)^2 / \sum k \ge c' h^{-d}$, while the root again has mass at most $(2\pi v_0)^{-d/2}$. In both cases the root's weight is $O(h^d) = O(G^{-1})$.

Leaves form a kernel smoother. Among the leaves, $w_\gamma \propto k_\gamma^r$ with $r = 1$ (LIK) or $r = 2$ (PATH). For any $\rho > 0$, splitting the exponent and using the lower bound on $\max_\gamma k_\gamma^r$ for the normaliser,

$$\sum_{\|\gamma - x\| > \rho} w_\gamma \;\le\; \frac{e^{-r\rho^2/4h^2} \sum_\gamma e^{-r\|x-\gamma\|^2/4h^2}}{e^{-rd\beta^2/8}} \;\le\; C'' e^{-r\rho^2/4h^2},$$

so $|\sum_\gamma w_\gamma g(\gamma) - g(x)| \le \omega_g(\rho) + 2\|g\|_\infty C'' e^{-\rho^2/4h^2}$. Taking $\rho = \sqrt{h}$ makes this vanish as $h \to 0$, uniformly in x; adding the root's $O(h^d)$ weight (times $2\|g\|_\infty$) gives the first claim. For Lipschitz g bound the leaves' part directly: $|\sum_\gamma w_\gamma g(\gamma) - g(x)| \le LD \sum_\gamma w_\gamma \|\gamma - x\| \le C\, LD\, h$, because

$\sum_\gamma e^{-r\|x-\gamma\|^2/2h^2}\|x-\gamma\| \le C(\beta,d)\,h$ by the same comparison with a Gaussian sum and the normaliser is bounded below as above; with $h \asymp G^{-1/d}$ and the root's weight $O(G^{-1})$ this is (4). The leaves-only variant drops the root term; restricted to the leaves of this tree, PRIOR coincides with LIK (equal counts) and POSTERIOR weights the leaves by $k_\gamma/\sum k$, i.e. $r = 1$. □

On the same flat tree the PRIOR weighting behaves differently: it multiplies each concept's likelihood by its *relative count*, and the root then keeps a share of the mixture that does not vanish as the variance floor shrinks.

Proposition B.3 (On a flat tree the prior- and posterior-weighted all-concept mixtures keep a fixed bias)**.** *In the setting of the proof of Theorem B.2, with PRIOR weighting and full expansion, the root's weight is* $\lambda(x) = \mathcal{N}_{root}(x)/(\mathcal{N}_{root}(x) + \theta_h(x))$*, where* $\mathcal{N}_{root}(x) = \mathcal{N}(x;\mu_{root},v_{root})$ *and* $\theta_h(x) = G^{-1}\sum_\gamma k_\gamma$ *is bounded above by a constant* $C(\beta,d)$ *independent of* h*. Hence* $\lambda(x) \ge \mathcal{N}_{root}(x)/(\mathcal{N}_{root}(x)+C(\beta,d))$ *for all* h*, and* $\hat{y}(x) - g(x) = \lambda(x)\big(\mu_{root,Y} - g(x)\big) + o(1)$*: the error does not vanish wherever* $g(x)$ *differs from the grid mean of* g*. With POSTERIOR weighting the leaves' masses* e^{π_γ} *sum to exactly one, so* $\lambda(x) = \mathcal{N}_{root}(x)/(\mathcal{N}_{root}(x)+1)$ *and the same conclusion holds.*

Proof. Leaf γ has prior $1/G$, so the leaves' mass is $\theta_h(x) = G^{-1}\sum_\gamma k_\gamma = \beta^d(2\pi)^{-d/2}\sum_\gamma e^{-\|x-\gamma\|^2/2h^2}$ (using $G^{-1} = \delta^d$), a Riemann sum of mesh β of the standard Gaussian density, bounded by $C(\beta,d)$ and bounded below by a positive constant (it is close to 1 in the interior when β is small, but it does not converge as $h \to 0$; it oscillates with x/h). The root has prior 1 and mass $\mathcal{N}_{\text{root}}(x)$, which tends to a positive limit. The leaves' mixture converges to $g(x)$ as in Theorem B.2. Under POSTERIOR the leaves are the root's children and their path posteriors are the sibling-normalised posteriors, which sum to one, while the root is scored by its likelihood (Definition A.3). □

Remark B.4 (A hierarchy changes the picture). The bias of Proposition B.3 belongs to the flat tree, not to the weightings alone. In a hierarchy each level of concepts carries a bounded share of the PRIOR mixture, so the root and the coarse levels together receive a share of order 1/depth: on a balanced binary tree over a grid of $G = 2^D$ points the PRIOR all-concept error falls roughly like $1/D$, i.e. like $1/\log G$, and in learned trees it falls like $1/\log N$ (Table 4). POSTERIOR behaves worse: soft sibling posteriors at the coarse splits leak mass across the split boundaries, and on the same binary tree its error stays at a floor even when the mixture is restricted to the leaves (Appendix G). The LIK and count-weighted leaves-only mixtures converge in both trees.

B.3 Composition: the Cobweb–Kolmogorov–Arnold theorem

Lemma B.5 (Approximating a DAG of continuous maps)**.** *Let* D *be a finite DAG of continuous maps* $g_v : \mathbb{R}^{k_v} \to \mathbb{R}$ *whose inputs are raw coordinates of* $x \in [0,1]^n$ *or outputs of other vertices, and let* F *be the map it computes; let* K_v *be the (compact) set of input vectors of* v *as* x *ranges over* $[0,1]^n$*. For every* $\varepsilon > 0$ *there are* $\rho_v, \eta_v > 0$ *such that any maps* $\hat{g}_v$ *with* $\sup_{K_v+\bar{B}_{\rho_v}} |\hat{g}_v - g_v| \le \eta_v$ *define a network* $\hat{F}$ *with* $\sup_x |\hat{F}(x) - F(x)| \le \varepsilon$*.*

Proof. Choose the tolerances in reverse topological order. At the output vertex put $\eta_o = \varepsilon/2$ and choose $\rho_o \le 1$ with $\omega_{g_o}(\rho_o) \le \varepsilon/2$ on the compact $K_o + \bar{B}_1$. Given the tolerance $\rho_v \le 1$ by which the input vector of v may deviate, allow each of its k_v inputs to deviate by $\rho_v/\sqrt{k_v}$: for each incoming vertex u set $\eta_u = \rho_v/(2\sqrt{k_v})$ and choose $\rho_u \le 1$ with $\omega_{g_u}(\rho_u) \le \rho_v/(2\sqrt{k_v})$ on $K_u + \bar{B}_1$, taking minima when u feeds several vertices. Then, in topological order, every vertex's input deviates from its exact value by at most ρ_v, so its output deviates by at most $\eta_v + \omega_{g_v}(\rho_v)$, which is within its consumers' tolerance; at the output the deviation is at most ε. □

Theorem B.6 (Cobweb–Kolmogorov–Arnold)**.** *Let $n \geq 2$. For every $f \in C([0,1]^n)$ and $\varepsilon > 0$ there is a Cobweb forest with activation messages, in which every tree has at most two inputs, consisting of*

- *$n(2n+1)$ univariate trees for the inner functions $\varphi_{q,p}$ (the functions do not depend on f; the resolution their trees need does),*
- *$(2n+1)(n-1)$ two-input trees forming the inner sums,*
- *$2n+1$ univariate trees for the outer functions Φ_q, and*
- *$2n$ two-input trees forming the outer sum,*

$4n^2+4n$ trees in all, whose output is within ε of f on $[0,1]^n$.

Proof. Write f in the form (1) and read it as a DAG of continuous maps of one or two variables: $\varphi_{q,p}$, the binary additions $(a,b) \mapsto a+b$ chained to form the $2n+1$ inner sums of n terms and the outer sum of $2n+1$ terms, and Φ_q. Apply Lemma B.5, and realise each $\hat{g}_v$ on the compact $K_v + \bar{B}_{\rho_v}$ by Theorem B.2 as a tree over instances $(\text{inputs}, g_v(\text{inputs}))$; its activation is the required $\hat{g}_v$. The count is $n(2n+1)+(2n+1)(n-1)+(2n+1)+2n = 4n^2+4n$. □

Proposition B.7 (Two inputs are necessary)**.** *If every tree of a forest has fan-in at most one, its output depends on at most one raw coordinate. Hence fan-in two is the least fan-in at which Cobweb forests are universal.*

Proof. By induction in topological order, each tree's messages are functions of the at most one raw coordinate at the source of its unique input chain. □

Remark B.8 (Rates and smooth KA representations). Theorem B.6 uses a fixed number of trees, but for an arbitrary f the inner functions of (1) are rough, and the number of leaves each tree needs is not controlled. As for KANs [14, Thm. 2.1], the useful statement assumes structure. If f is computed by a DAG of s maps of fan-in $\leq k$, each L-Lipschitz on a neighbourhood of its input set, then (4) and the stability argument of Lemma B.5 give a forest of s trees with G leaves each and error $C\,G^{-1/k}$, with C depending on s, k, L and the diameters of the input sets—a rate set by the fan-in k, not by n. A single tree over the n inputs has rate $G^{-1/n}$ (Theorem B.2). Cobweb trees realise sums as learned two-input maps; KANs sum at nodes for free, which is why their rate is governed by one-dimensional splines.

B.4 Label forests

Theorem B.9 (Label forests are universal)**.** *For every $f \in C([0,1]^n)$ and $\varepsilon > 0$ there is a forest of n univariate trees without outcomes that pass leaf labels to an output tree whose instances consist of the n labels (nominal attributes) and the target, such that the output tree's prediction (LIK, leaves only, full expansion) is uniformly within ε of f.*

Proof. Let tree p be the depth-one tree over the K grid points of spacing $\delta = 1/K$ in $[0,1]$, all with count one and variance h^2. Its label for x_p is $\arg\max_\gamma\{\log n_\gamma + \ell_\gamma(x_p)\}$, the nearest grid point, so the labels partition $[0,1]^n$ into boxes B_t of side δ, indexed by label tuples t. Let the output tree be the depth-one tree with one leaf per tuple t, holding the instance $(t, f(\text{centre}(B_t)))$. For a query with tuple t, a leaf t' that disagrees with t in m attributes has likelihood ratio $\big(\alpha/(1+\alpha)\big)^m$ relative to leaf t (Definition A.1, counts one). There are at most $\binom{n}{m}(K-1)^m$ such leaves, so their total weight is at most $\sum_{m\geq 1}\binom{n}{m}(K\alpha)^m = (1+K\alpha)^n - 1$. Hence $|\hat{y}(x) - f(x)| \leq \omega_f(\sqrt{n}\,\delta/2) + 2\|f\|_\infty\big((1+K\alpha)^n - 1\big)$, which is below ε for K large and then α small. □

The label forest of Theorem B.9 has an output tree with K^n leaves: labels carry no metric, so this construction pays for every cell of the product partition. Activations carry the metric and allow the fan-in-two construction of Theorem B.6.

Remark B.10 (Two elementary routes to universality). There are two short arguments that a single tree can represent any function, and the results above make each precise. The first is retrieval: store an example at every input on a fine grid, quantize the inputs, and predict the stored output of the query's cell—a nominal tree of grid cells does this, which is the construction of Theorem B.9. The second is mixtures: mixtures of enough Gaussians approximate any continuous density, and a tree's concepts are such a mixture. Approximating the joint density of (x, y) does not by itself make the *prediction* of y uniformly good, because the conditional mean divides by the marginal density of x, which can be small; Theorem B.2 therefore proves the regression statement directly, for the mixture weights the implementation actually uses, and Proposition B.3 shows that two of those weightings fail it on a flat tree.

B.5 Interweaving trees

Proposition B.11 (Merging trees under a common root gives mixtures). *Let $\mathcal{T}_1, \dots, \mathcal{T}_m$ be trees over the same instance space with the same value sets (e.g. continuous-only trees), and let $\mathcal{T}$ be the tree whose root pools their roots' statistics and has $\mathcal{T}_1, \dots, \mathcal{T}_m$ as its subtrees. Under full expansion with* LIK *or* PATH *weighting,*

$$\hat{y}_{\mathcal{T}}(q) = \lambda_0(q)\, \mu_{root} + \sum_{j=1}^{m} \lambda_j(q)\, \hat{y}_{\mathcal{T}_j}(q), \qquad \lambda_j \geq 0,\ \sum_{j=0}^{m} \lambda_j = 1,$$

with $\lambda_0 \propto e^{\ell_{root}}$ and $\lambda_j \propto Z_j$ (LIK) or $\lambda_j \propto P(root_j \mid q)\, Z_j$ (PATH), where Z_j is the total unnormalised weight of $\mathcal{T}_j$'s own mixture and $P(root_j \mid q)$ the sibling-normalised posterior of $\mathcal{T}_j$'s root.

Proof. Under LIK a concept's score does not depend on the tree it sits in, so the mixture over $\mathcal{T}$ splits into the root term and the m subtree mixtures. Under PATH every concept of $\mathcal{T}_j$ gains the same term $\log P(\text{root}_j \mid q)$ in π_c (the sibling sets below root_j are unchanged), which rescales the subtree's weights uniformly and leaves its internal mixture unchanged. With different value sets the smoothing denominators K_A change, and the identity holds only approximately. □

Interweaving therefore closes the class of tree predictors under input-dependent convex combinations; it never composes them. Composition—one tree reading another's representation—is what forests add.

C Learning one tree: statements and proofs

Lemma C.1 (Leaf preservation). *Run* `ifit` *on any sequence $z_1, \dots, z_N$ with any choices of operators (including add-only training), and suppose the depth limit is never reached. Then (i) every concept's statistics are the pooled statistics of the instances stored in the leaves below it; (ii) no leaf is ever deleted; (iii) every instance is stored in exactly one leaf, and a leaf stores an arriving instance only if the leaf's empirical variance is within the exact-match tolerance τ and the instance is within τ of the leaf's current mean (coordinatewise; nominal parts must have the same value proportions, within τ). With $\tau = 0$ a leaf stores copies of one instance, and if the instances are pairwise distinct the leaves are in bijection with them. (The implementation uses $\tau = 10^{-5}$ per coordinate.)*

Proof. By induction over the branches of the sorting loop. BEST increments the current node and descends; NEW increments the current node and creates a leaf holding z; MERGE inserts a new internal node whose statistics are the pooled statistics of the two children it adopts; SPLIT deletes the best child, which is internal (the operator requires it to have at least two children), and promotes its children; at a leaf, an exact match increments it and any other instance triggers a fringe split, which copies the leaf's statistics into a new internal parent, keeps the old leaf, and adds a leaf for z. No branch deletes a leaf, every instance ends in exactly one leaf, and a leaf only absorbs exact matches. Two copies of the same instance need not share a leaf: sorting may lead the second copy to a different leaf, which it then splits. □

When the depth limit is reached the implementation inserts the instance into the current node, which may be a leaf; (iii) then fails, which is why the lemma assumes the default limit (1000) is not reached.

Theorem C.2 (A learned tree is universally consistent)**.** *Let (X_i, Y_i) be i.i.d. with $X \in \mathbb{R}^d$, $|Y| \le B$, $m(x) = \mathbb{E}[Y \mid X = x]$. Let $\mathcal{T}_N$ be the tree that Cobweb learns from $(X_i, Y_i)_{i \le N}$ in any order and with any operator choices, with exact matching ($\tau = 0$) and without reaching the depth limit, and let it predict Y from X with the* count-weighted *leaves-only mixture (*PRIOR *weighting restricted to the leaves) under full expansion, with variance floor h_N^2. If $h_N \to 0$ and $N h_N^d \to \infty$, then $\int |\hat{y}_{\mathcal{T}_N}(x) - m(x)|\, \mu(dx) \to 0$ almost surely, whatever structure category utility builds (μ the distribution of X).*

Proof. By Lemma C.1 with $\tau = 0$, every instance is stored in exactly one leaf and a leaf c stores n_c copies of one instance (x_c, y_c), with zero spread. The PRIOR score of leaf c for a query x with Y missing is $\log \mathcal{N}(x; x_c, h_N^2 I) + \log(n_c/N)$, so the count-weighted leaves-only mixture is $\sum_c n_c K_{h_N}(x - x_c) y_c / \sum_c n_c K_{h_N}(x - x_c) = \sum_i K_{h_N}(x - X_i) Y_i / \sum_i K_{h_N}(x - X_i)$, the Nadaraya–Watson estimate with a Gaussian kernel. The Gaussian kernel is *regular* (it is bounded below by a positive constant on a ball and $\int \sup_{\|u-x\| \le r} K(u)\, dx < \infty$), and for regular kernels and bounded Y this estimate is strongly consistent in $L_1(\mu)$ under $h_N \to 0$, $N h_N^d \to \infty$ [8]; see Györfi et al. [12, Ch. 5] for the boxed-kernel version in L_2. □

Remark C.3 (Counts matter)*.* The leaves-only LIK mixture ignores counts. It equals the count-weighted mixture when every leaf stores one instance—for instance when X has a density and matching is exact—but not when instances repeat, exactly (atoms in a nominal input or in the target) or within the implementation's tolerance $\tau = 10^{-5}$: it then averages the distinct stored values instead of the samples. For a Bernoulli target whose copies share one leaf per value in each cell it gives $1/2$ in every cell as $\alpha \to 0$, whatever the cell's frequency (Appendix G). With $\tau > 0$ a leaf compares each arriving instance with its current mean, so the stored instances can drift from the leaf's mean by more than τ—of order $\tau \log n_c$ for a leaf with n_c copies; the count-weighted mixture is then the Nadaraya–Watson estimate on a design perturbed by that much. Since τ is fixed while $h_N \to 0$, Theorem C.2 describes the implementation only while $h_N \gg \tau \log N$, a pre-asymptotic regime that covers every experiment here except the smallest variance floors of Fig. 5. The PATH mixture of the experiments weights leaves by their path posteriors, which contain the counts.

Remark C.4 (What the structure is for)*.* Theorem C.2 holds because Cobweb never forgets an instance; the hierarchy plays no role in it. The structure matters for three things the theorem does not cover: (a) a *budget* $M < \infty$, where best-first search must find the leaves near the query; (b) the PATH weighting, where the mixture weight of a leaf is its path posterior times its likelihood, so the leaves-only mixture is not a kernel smoother in a learned tree (Lemma B.1 still bounds its

error); and (c) the *labels* a tree passes to other trees, which are its concepts. The experiments use the implementation's default, PATH with a leaf budget; they are therefore a test of these structural effects as much as of the theorem. The branch-and-bound stop of `categorize(use_best=True)` ends the whole search as soon as the best score exceeds the path posterior of the concept just expanded. This is sound only if no concept still in the queue has a larger path posterior, but the queue is ordered by score, not by path posterior; and with Gaussian densities scores can exceed path posteriors because $\ell_c > 0$. The stop can therefore return a concept whose score is below the best, with continuous attributes and with nominal ones alike (Appendix G exhibits both). A sound stop must bound what a descendant can still score: with $\ell_{\max}$ an upper bound on any concept's log-likelihood (0 for nominal attributes, $-\frac{|O|}{2}\log(2\pi h^2)$ for $|O|$ observed Gaussian coordinates), a subtree whose root has $\pi_c + \ell_{\max}$ below the best score found so far can be pruned.

D Tasks: statements and proofs

A Cobweb tree is unsupervised: it models the joint distribution of whatever its instances contain and predicts any attribute that is missing from a query. What a tree *passes on* is therefore fixed by what its instances contain. We make this the central object.

D.1 Tasks as inputs and outcomes

Definition D.1 (Cognitive task)**.** Let the stream consist of i.i.d. observations (X, Y), $X \in \mathbb{R}^n$ the raw input and Y the target. A *cognitive task* for a tree τ is a pair (C_τ, V_τ):

- the *input* C_τ, what the tree perceives: its raw coordinates X_{S_τ} and the messages of its parents;
- the *outcome* V_τ, what the tree predicts and later observes: either nothing ($V_\tau = \varnothing$) or an *observable* variable, i.e. coordinates of the observation that the input does not contain—the target Y, or raw coordinates X_J with $J \cap S_\tau = \emptyset$.

The tree learns the instances (C_τ, V_τ) with `ifit` and is queried with V_τ missing (Definition A.4). A *task assignment* A gives every tree of a forest a task; the output tree's outcome is Y.

Three families of tasks occur in the systems that motivate this paper. *Categorization* ($V_\tau = \varnothing$): the tree passes only labels, as the filter hierarchy of Convolutional Cobweb does. *Funnel* ($V_\tau = Y$): every tree predicts the final target, as every tree of the Cobweb language model predicts the next word. *Cross-view* ($V_\tau = X_j$, $j \notin S_\tau$): the tree predicts a part of the input it does not perceive—a neighbouring pixel, the next token, the context of a chunk, as TRELLIS's content hierarchy models its context. The input–outcome reading is the one of associative learning, in which an organism learns to predict an outcome it will observe from the inputs it perceives [20]; the online protocol of Definition A.6 is literally "predict, then observe, then learn".

Observability is the constraint that separates learnable from representable forests.

Remark D.2 (The KA tasks are not observable)*.* In Theorem B.6 the tree for Φ_q holds the values $\Phi_q(u_q)$. The stream never shows them: it shows only their sum $Y = \sum_q \Phi_q(u_q)$. A KAN separates the terms of the sum by back-propagating the error of the sum through each of them; a Cobweb tree can only predict what it will observe. The inner functions $\varphi_{q,p}$ do not depend on f, so a forest could be *given* them as fixed tasks, but the outer decomposition is not identifiable from the stream without further structure. The rest of this section asks which *observable* tasks recover f anyway.

D.2 What a learned forest converges to

Definition D.3 (Ideal forest, sufficiency, deficit)**.** Let A be a task assignment of an activation forest. Define, in topological order, the *ideal input* and *ideal message* of each tree,

$$U_\tau = \big(X_{S_\tau}, (m_\sigma(X))_{\sigma\to\tau}\big), \qquad m_\tau(X) = \mathbb{E}[V_\tau \mid U_\tau] = \psi_\tau(U_\tau),$$

and the *ideal output* $F_A(X) = m_o(X) = \mathbb{E}[Y \mid U_o]$. Let $f(X) = \mathbb{E}[Y \mid X]$. The assignment is *sufficient* if $F_A = f$ almost surely, and its *deficit* is

$$\Delta(A) = \mathbb{E}\big[(f(X) - F_A(X))^2\big] = \mathbb{E}\,\mathrm{Var}(Y \mid U_o) - \mathbb{E}\,\mathrm{Var}(Y \mid X).$$

The identity holds because U_o is a function of X, so $F_A = \mathbb{E}[f(X) \mid U_o]$ and $\mathrm{Var}(Y \mid U_o) = \mathbb{E}[\mathrm{Var}(Y \mid X) \mid U_o] + \mathrm{Var}(f(X) \mid U_o)$.

Assumption D.4 (Regularity)**.** (A1) The observations are i.i.d., X has compact support, and every outcome is bounded, $|V_\tau| \le B$. (A2) For every tree, the ideal input $U_\tau \in \mathbb{R}^{k_\tau}$ has a density bounded away from zero and infinity on its support D_τ, a compact set with Lipschitz boundary, and ψ_τ extends to a uniformly continuous function on a neighbourhood of D_τ. (A3) Every tree matches instances exactly ($\tau = 0$) and predicts with the count-weighted leaves-only mixture under full expansion, with variance floor $h_{\tau,N}^2$, $h_{\tau,N} \to 0$ and $N h_{\tau,N}^{k_\tau}/\log N \to \infty$. (A4) For every edge $\sigma \to \tau$, $\varepsilon_{\sigma,N}/h_{\tau,N} \to 0$ in probability, where $\varepsilon_{\sigma,N} = \sup_x |\hat m_\sigma(x) - m_\sigma(x)|$ is the parent's uniform error.

(A4) is a condition on the schedule: by induction the parents' errors tend to zero (Theorem D.5), so variance floors that shrink slowly enough satisfy it. (A2) is restrictive: an ideal message with a critical point, such as $\mathbb{E}[Y \mid X_2] \propto e^{X_2^2}$ at $X_2 = 0$, has an unbounded density there, so the funnel forests of the KAN examples are outside Theorem D.5; the experiments test them anyway.

Theorem D.5 (A Cobweb-learned forest converges to its ideal forest)**.** *Let $\mathfrak{F}$ be an activation forest with task assignment A, trained with the split protocol on N observations, and let Assumption D.4 hold. Then for every tree τ, $\sup_x |\hat m_\tau(x) - m_\tau(x)| \to 0$ in probability; in particular the output satisfies $\sup_x |\hat F_N(x) - F_A(x)| \to 0$ in probability.*

Proof. Induction over the levels of the DAG. Fix a tree τ at level ℓ and condition on the chunks of the earlier levels, on which its parents were trained and frozen. Write $\hat u_\tau(x) = (x_{S_\tau}, (\hat m_\sigma(x))_\sigma)$ for its learned input map and $u_\tau(x)$ for the ideal one, so $\|\hat u_\tau - u_\tau\|_\infty \le \varepsilon := \sqrt{k_\tau}\max_\sigma \varepsilon_{\sigma,N}$.

Step 1: the tree is a kernel smoother. The tree learns the instances $(\hat U_i, V_i) = (\hat u_\tau(X_i), V_{\tau,i})$ of chunk ℓ, which are i.i.d. given the earlier chunks. By Lemma C.1 its leaves store copies of single instances, with zero spread, and, as in the proof of Theorem C.2, its count-weighted leaves-only activation at a input u is the Nadaraya–Watson estimate $\hat\psi(u) = \sum_i K_h(u - \hat U_i)V_i / \sum_i K_h(u - \hat U_i)$ over the instances of the chunk, with the Gaussian kernel of bandwidth $h = h_{\tau,N}$. (With the implementation's tolerance the stored inputs move by at most about $\tau(1 + \ln N)$ (Remark C.3), a perturbation that Step 2 absorbs when $\tau \ln N = o(h)$.)

Step 2: the perturbed design does not matter. Let $\check\psi$ be the same estimate built on the ideal inputs $U_i = u_\tau(X_i)$. We take as given the uniform Nadaraya–Watson law that, under (A1)–(A3), $\check\psi \to \psi_\tau$ uniformly on the δ-neighbourhood of D_τ for any $\delta = o(h)$, in probability. The cited results [7, 16] state it on interior compact sets; up to a Lipschitz boundary and for the Gaussian kernel it follows from the cone condition (a fixed fraction of every small ball around a point of D_τ lies in D_τ), the uniform kernel-density law of Einmahl and Mason [9] for the denominator, and the

bias bound $\omega_{\psi_\tau}(Rh) + 2Be^{-R^2/16}$, but we do not write out that derivation. Write $\hat{U}_i = U_i + \Delta_i$, $\|\Delta_i\| \le \varepsilon$. For $\|u - U_i\| \le Rh$,

$$\frac{K_h(u-\hat{U}_i)}{K_h(u-U_i)} = \exp\Big(\frac{2\langle u-U_i, \Delta_i\rangle - \|\Delta_i\|^2}{2h^2}\Big) \in \big[e^{-\eta}, e^{\eta}\big], \qquad \eta = \frac{R\varepsilon}{h} + \frac{\varepsilon^2}{2h^2},$$

and for $\|u - U_i\| > Rh \ge 2\varepsilon$, $K_h(u - \hat{U}_i) \le (2\pi h^2)^{-k/2} e^{-\|u-U_i\|^2/8h^2}$. Since the density of U_τ is bounded below on D_τ and bounded above, and D_τ has Lipschitz boundary, the kernel mass of the near points is at least $c\,N$ and that of the far points at most $C\,Ne^{-R^2/16}$, uniformly in u within $o(h)$ of D_τ, with probability tending to one, by the uniform consistency of kernel density estimates [9]. Hence $\sup_u |\hat{\psi}(u) - \check{\psi}(u)| \le 2B\big(e^{2\eta} - 1 + C'e^{-R^2/16}\big)$, which tends to zero in probability when $R = R_N \to \infty$ slowly enough that $R\varepsilon/h \to 0$, possible by (A4).

Step 3: the test-time input. At test time the tree is queried at $\hat{u}_\tau(x)$, so $|\hat{m}_\tau(x) - m_\tau(x)| \le \sup_{D_\tau^{(\varepsilon)}} |\hat{\psi} - \psi_\tau| + \omega_{\psi_\tau}(\varepsilon)$, where $D_\tau^{(\varepsilon)}$ is the ε-neighbourhood of D_τ; both terms tend to zero in probability by Steps 1–2 and (A2). This is the induction hypothesis for the children of τ; at the output tree it is the claim. □

Corollary D.6 (Consistency if and only if sufficiency)**.** *Under the hypotheses of Theorem D.5,* $\mathbb{E}[(\hat{F}_N(X) - f(X))^2] \to \Delta(A)$. *The forest is consistent for f if and only if its task assignment is sufficient.*

Proof. $\hat{F}_N$ and f are bounded by B, so uniform convergence in probability gives $\mathbb{E}(\hat{F}_N - F_A)^2 \to 0$, and $\mathbb{E}(\hat{F}_N - f)^2 \to \mathbb{E}(F_A - f)^2 = \Delta(A)$. □

Theorem D.5 separates what Cobweb's learning does from what the tasks determine: every tree learns the conditional expectation of its outcome given its input, nothing more and nothing less; whether the composition of these conditional expectations is f is a property of the task assignment alone.

D.3 Which tasks are sufficient

Funnel tasks. A convenient sufficient condition for funnel forests is that each tree's ideal message equals $\mathbb{E}[Y \mid X_{R(\tau)}]$, where $R(\tau)$ is the set of raw coordinates upstream of τ; this holds by induction when every $\mathbb{E}[Y \mid X_{R(\tau)}]$ is a function of the parents' $\mathbb{E}[Y \mid X_{R(\sigma)}]$ and of X_{S_τ}.

Proposition D.7 (Funnel tasks are sufficient for monotone-additive targets)**.** *Let X have compact support and consist of independent blocks $X_{B_1}, \dots, X_{B_m}$, and let $Y = \Phi\big(\sum_j g_j(X_{B_j})\big) + \xi$ with Φ continuous and strictly increasing, g_j continuous, and ξ centred and independent of X. Then every funnel activation forest whose graph is a rooted tree in which each block is the raw input of exactly one tree is sufficient.*

Proof. For an upstream set of blocks S, $\mathbb{E}[Y \mid X_S] = \mathbb{E}[\Phi(s_S + Z_{\bar{S}})] =: H_S(s_S)$ with $s_S = \sum_{j\in S} g_j(X_{B_j})$ and $Z_{\bar{S}}$ the sum over the other blocks, independent of X_S; H_S is continuous and strictly increasing because Φ is. A tree whose children cover the disjoint block sets $S_1, \dots, S_k$ and which reads the raw blocks S_0 has $s_S = \sum_i H_{S_i}^{-1}(m_{S_i}) + \sum_{j\in S_0} g_j(X_{B_j})$, so its $\mathbb{E}[Y \mid X_S] = H_S(s_S)$ is a continuous function of its parents' messages and its raw input. □

The KAN examples $\exp(\sin \pi x_1 + x_2^2)$, $\exp\big(\frac{1}{100}\sum_i \sin^2(\pi x_i/2)\big)$ and $\exp\big(\frac{1}{2}(\sin \pi(x_1^2 + x_2^2) + \sin \pi(x_3^2 + x_4^2))\big)$ (the last with the blocks (x_1, x_2) and (x_3, x_4)) are in this class.

Proposition D.8 (Funnel tasks can fail)**.** *Let X_1, X_2 be independent and symmetric about 0, with densities bounded away from zero and infinity on compact intervals, and $Y = X_1X_2$. For the funnel forest whose output tree reads the activations of a tree perceiving X_1 and a tree perceiving X_2, the ideal messages are $m_1 = m_2 \equiv 0$, so $F_A \equiv 0$ and $\Delta(A) = \mathrm{Var}(Y)$. If the trees are learned as in Theorem D.5, with variance floors satisfying (A3) and, for the output tree, (A4), the learned output converges uniformly in probability to $\mathbb{E}Y = 0$: in the limit the forest learns nothing, whatever structure its trees build.*

Proof. $\mathbb{E}[X_1X_2 \mid X_1] = X_1\mathbb{E}X_2 = 0$, and likewise for X_2, so the output tree's ideal input is the constant $u_0 = (0, 0)$. The source trees satisfy Assumption D.4, so their activations converge to 0 uniformly (Steps 1–3 of the proof of Theorem D.5). The output tree therefore learns and is queried at inputs within $\varepsilon \to 0$ of u_0; all its kernel weights lie within the factor $e^{-2\varepsilon^2/h^2} \to 1$ of one another, so its prediction converges to the mean of Y over its chunk, and that mean to $\mathbb{E}Y = 0$. At finite N the activations are noisy functions of the coordinates and can carry a little information; the statement is asymptotic. □

Cross-view tasks. A tree that predicts another part of the input never sees the target. It recovers what the views share, which suffices when the target depends on the input only through what the views share—the multi-view redundancy assumption of co-training and of contrastive learning [2, 24].

Proposition D.9 (Cross-view tasks are sufficient under multi-view redundancy)**.** *Let $Z \in \mathbb{R}^r$ be latent and $X = (X^{(1)}, \dots, X^{(m)})$ consist of $m \geq 2$ views with (Z, X) jointly Gaussian and centred, $X^{(v)} = \Lambda_v Z + E_v$, $\mathrm{Cov}(Z)$ and every $\mathrm{Cov}(E_v)$ nonsingular, the noises E_v independent of each other and of Z. Let $Y = g(Z) + \xi$ with $\mathbb{E}|g(Z)| < \infty$ and ξ centred and independent of (Z, X). Give the tree of view v the input $X^{(v)}$ and as outcome r coordinates W_v of other views whose loading matrix $\Lambda_{W_v} \in \mathbb{R}^{r\times r}$ is invertible, and let the output tree have the activations of the m view trees as input and Y as outcome. This assignment is sufficient, and no tree but the output tree perceives Y.*

Proof. Write $\mu_v = \mathbb{E}[Z \mid X^{(v)}]$ and P_v for the posterior precision given $X^{(v)}$, and P_0 for the prior precision; both precisions are constants. Since $W_v = \Lambda_{W_v} Z + E'$ with E' independent of $(Z, X^{(v)})$, the ideal message is $m_v = \mathbb{E}[W_v \mid X^{(v)}] = \Lambda_{W_v}\mu_v$, an invertible linear function of μ_v. By conditional independence of the views, $p(z \mid x) \propto p(z)\prod_v p(x^{(v)} \mid z) \propto p(z)^{1-m}\prod_v p(z \mid x^{(v)})$, a Gaussian with precision $P = \sum_v P_v - (m-1)P_0$ and mean $P^{-1}\sum_v P_v\mu_v$. The posterior of Z given X therefore depends on X only through $(\mu_v)_v$, hence through $(m_v)_v$, and so does $f(X) = \mathbb{E}[g(Z) \mid X]$. Thus $F_A = \mathbb{E}[Y \mid (m_v)_v] = f$. □

The assumption is necessary in the following sense: if the target carries a view-specific component, e.g. $Y = e^\top E_1$ with $X^{(1)}$ of dimension larger than r and e generic, the cross-view messages discard it and the deficit is positive. In images and text the views are patches and contexts, and the cross-view outcome is a neighbouring pixel or the next token.

Categorization. A tree without an outcome passes labels. Its labels do not depend on f at all, which is their strength and their cost.

Theorem D.10 (Consistency of label forests)**.** *Let a forest consist of n trees without outcomes over the raw coordinates passing labels at depth k_N (or leaf labels) to an output tree that models (labels, Y), $|Y| \leq B$, learned with the split protocol; let the output tree match instances exactly and predict with the count-weighted leaves-only mixture under full expansion. Let X have compact*

support $\mathcal{X}$, let $\mathcal{P}_N$ be the product partition induced by the frozen label maps, and let M_N be the number of its cells that meet $\mathcal{X}$. If, in probability, (a) $\max_C \operatorname{diam}(C \cap \mathcal{X}) \to 0$ over the cells C meeting $\mathcal{X}$ (the label maps partition all of the input space, so cells at the edge are unbounded; only their part in $\mathcal{X}$ matters), (b) $M_N/N \to 0$, and (c) $\alpha_N M_N \to 0$, then the forest's prediction $\hat{y}$ satisfies $\mathbb{E}\int(\hat{y}-m)^2\,d\mu \to 0$.

Proof. Condition on the frozen partition and let $N' \asymp N$ be the size of the output chunk, n_C the number of its points in cell C, and $\hat{m}$ the partitioning estimate ($\hat{m} = \bar{y}_C$, the mean of the chunk's Y in C, and $\hat{m} = 0$ on empty cells). Under (a) and (b) the proof of Györfi et al. [12, Thm. 4.2], applied to the fixed partition (its conditions are stated for the cells meeting a ball, which here may be taken to contain $\mathcal{X}$), gives $\mathbb{E}\int(\hat{m}-m)^2 d\mu \to 0$; it remains to show $\mathbb{E}\int(\hat{y}-\hat{m})^2 d\mu \to 0$.

Take a query in a cell C with $n_C \geq 1$ and label tuple t_C. By Lemma C.1 ($\tau = 0$) every leaf c stores n_c copies of one instance (t_c, y_c), and the leaves with $t_c = t_C$ store exactly the n_C points of C. With K_a labels for tree a, an attribute that a leaf matches has probability $(n_c+\alpha)/(n_c+(K_a+1)\alpha) \in [\kappa_a, 1]$ with $\kappa_a = (1+(K_a+1)\alpha)^{-1}$, and one it does not match has probability $\alpha/(n_c+(K_a+1)\alpha) \leq \alpha/n_c$. Hence, with $\kappa = \prod_a \kappa_a$, the matching leaves carry count-weighted mass between κn_C and n_C, distributed in proportion to their counts up to factors in $[\kappa, 1]$, and every other leaf carries at most $n_c(\alpha/n_c)^m \leq \alpha$, where $m \geq 1$ is the number of attributes it misses. There are at most N' other leaves, so

$$|\hat{y} - \bar{y}_C| \;\leq\; 2B\Big(\min\Big(1, \frac{\alpha N'}{\kappa n_C}\Big) + \frac{1-\kappa}{\kappa}\Big),$$

and $|\hat{y}-\hat{m}| \leq B$ when $n_C = 0$. Every label of tree a is the label of some cell, so $K_a \leq M_N$ and $1-\kappa \leq \alpha\sum_a (K_a+1) \leq n\alpha(M_N+1) \to 0$ by (c). For a cell of probability $\mu(C)$, $\Pr(n_C < N'\mu(C)/2) \leq e^{-N'\mu(C)/8}$ (Chernoff), so

$$\mathbb{E}\int \min\Big(1, \frac{\alpha N'}{\kappa n_C}\Big) d\mu \;\leq\; \sum_{C\cap\mathcal{X}\neq\emptyset} \mu(C)\Big(e^{-N'\mu(C)/8} + \frac{2\alpha}{\kappa\,\mu(C)}\Big) \;\leq\; \frac{8M_N}{eN'} + \frac{2\alpha M_N}{\kappa},$$

using $te^{-N't/8} \leq 8/(eN')$; both terms tend to zero by (b) and (c). Therefore $\mathbb{E}\int(\hat{y}-\hat{m})^2 d\mu \to 0$. The bounds hold conditionally on the partition; since everything is bounded, the conditions in probability and bounded convergence give the claim. □

Remark D.11 (The trade-off). Condition (b) requires the product partition to have fewer occupied cells than samples; if X fills the K^n cells of K labels per tree, that is $K^n = o(N)$, so the rate of a label forest is governed by n, not by the fan-in. For funnel and cross-view forests Theorem D.5 gives consistency but no rate; the bounds of Remark B.8, which are for constructed trees, suggest rates governed by the fan-in, at the price of sufficiency (Proposition D.8).

Conjecture D.12 (Hybrid messages)**.** *A forest whose trees pass labels and funnel activations, and whose output tree's variance floor on the activation coordinates satisfies an (A3)–(A4)-type schedule, is consistent whenever its label cells satisfy the conditions of Theorem D.10 (through its labels), and attains the fan-in rate when its funnel assignment is sufficient (through its activations). (Without a schedule on the floor the conjecture fails: as $h_N \to 0$ quickly the output tree becomes a nearest-neighbour rule in activation space and ignores the labels.)*

Remark D.13 (When should a forest beat a single tree?). Comparing the upper bounds suggests where to look, without proving a separation. For a noise-free C^2 target on an i.i.d. design, a single Nadaraya–Watson tree's interior error is of order N^{-1} for $n = 1$, $N^{-1/2}$ for $n = 2$ and $N^{-2/(n+2)}$

for $n \geq 3$ (the design itself fluctuates; on a compact support the $O(h)$ bias in a boundary layer of width $O(h)$ lowers every exponent but leaves the comparison below unchanged); each tree of a funnel forest regresses an outcome that is noisy given its own input (the coordinates it does not perceive act as noise, of variance $\mathrm{Var}(Y \mid X_{R(\tau)})$), with error of order $N^{-2/(k+4)}$. The bounds favour the forest only when $n > k+2$. For a noisy target the single tree's bound is $N^{-2/(n+4)}$ and the comparison favours the forest as soon as $n > k$.

E Choosing tasks: statements and proofs

Corollary D.6 reduces learnability to choosing a sufficient task assignment. The assignment is part of the forest's structure; we ask whether it can be inferred from the stream, and by what rule.

E.1 Tasks cannot be inferred locally

Definition E.1 (Local task rule)**.** A *local task rule* decides something about a tree τ—keep or prune it, which task to give it, which message to pass—as a measurable function of τ's own training stream (the instances $(C_{\tau,i}, V_{\tau,i})$, $i \leq N$, for a candidate outcome V_τ, possibly $\varnothing$) and of independent randomness (in the implementation each tree of a forest has its own tie-breaking stream). The tree's own prequential score, its category utility and its learned concepts are all functions of this stream.

Theorem E.2 (Locality obstruction)**.** *Let X_1, X_2, X_3 be independent and uniform on $[-1,1]$, observe only (X_1, X_2, Y), and consider the two worlds*

$$P_a: \ Y = \mathrm{sign}(X_1 X_2), \qquad P_b: \ Y = \mathrm{sign}(X_1 X_3).$$

(i) For $j \in \{1,2\}$, the pair (X_j, Y) has the same law under P_a and P_b: X_j and Y are independent and Y is uniform on $\{-1,1\}$. Hence, for every candidate outcome, the training stream of a tree with input X_j has the same law in both worlds, and so does the output of every local task rule. (ii) Under P_a, $f(X) = Y$ and discarding X_2 loses everything ($\mathbb{E}\,\mathrm{Var}(Y \mid X_1) = 1$ while $\mathbb{E}\,\mathrm{Var}(Y \mid X) = 0$); under P_b, X_2 is irrelevant ($f(X) = \mathbb{E}[Y \mid X_1] = 0$). Consequently every local rule that decides whether to keep the tree perceiving X_2 errs with probability at least $1/2$ in one of the two worlds. (iii) Under P_a the funnel activations of both univariate trees vanish, so every funnel forest built on them has the maximal deficit $\Delta = \mathrm{Var}(Y) = 1$, whereas the label forest of Theorem D.10 and a single tree over (X_1, X_2) are sufficient.

Proof. (i) Under P_a, $Y = \mathrm{sign}(X_1)\,\mathrm{sign}(X_2)$; given X_2, the factor $\mathrm{sign}(X_1)$ is uniform on $\{-1,1\}$ and independent of X_2, so Y is uniform and independent of X_2; by symmetry the same holds for X_1. Under P_b, $Y = \mathrm{sign}(X_1)\,\mathrm{sign}(X_3)$ is independent of X_2 outright and, by the same argument, of X_1. The stream of a tree with input X_j is an i.i.d. sample of (X_j, V) with $V \in \{Y, \varnothing\}$ (or another observable whose joint law with X_j is the same in both worlds, such as X_{3-j}), so it has the same law in both worlds, and so does any measurable function of it. (Definition D.1 lets an outcome be Y or raw coordinates, not both; a tree whose stream contained X_j, X_{3-j} and Y together would see everything observed, and telling the worlds apart from that stream is composition.) (ii) Direct. If p is the probability that the rule keeps the tree, it errs with probability $1-p$ under P_a and p under P_b. (iii) $\mathbb{E}[Y \mid X_j] = 0$ for $j = 1,2$ by (i), so Proposition D.8's argument applies; the label and single-tree statements follow from Theorems C.2 and D.10. □

Corollary E.3 (Parity requires composition)**.** *Let $X_1, \ldots, X_n$ be independent and uniform on $[-1,1]$ and $Y = \prod_{j \in J} \mathrm{sign}(X_j)$. For every S that does not contain J, Y is independent of X_S.*

Hence no tree whose upstream raw set $R(\tau)$ misses a coordinate of J, and no rule using the streams of any set of trees whose inputs and outcomes together are functions of X_S with $J \not\subseteq S$, can detect the dependence; every funnel activation of a tree with $J \not\subseteq R(\tau)$ vanishes; and the ideal output of an activation forest whose outcomes are Y or raw coordinates equals f only if the raw input of some tree contains J (a merged input). Labels, which pass a tree's input rather than a prediction, are the other way to bring all of J to one tree; for learned forests these statements hold in the limit of Theorem D.5.

Proof. For $j \in J \setminus S$, $\operatorname{sign}(X_j)$ is uniform on $\{-1, 1\}$ and independent of X_S and of the other factors, so Y is uniform and independent of X_S. For the last claim, suppose no tree's raw input contains J and argue by induction in topological order that every ideal activation is constant. If a tree's parents' ideal activations are constant, its ideal input is a function of X_{S_τ} and its ideal message is $\mathbb{E}[V_\tau \mid X_{S_\tau}]$. For $V_\tau = Y$ this is $\mathbb{E}Y$, because $J \not\subseteq S_\tau$; for $V_\tau = X_j$ with $j \notin S_\tau$ it is $\mathbb{E}X_j$, because the coordinates are independent. Hence the output's ideal value is $\mathbb{E}[Y \mid X_{S_o}] = \mathbb{E}Y = 0$, while $f(X) = Y$ is not constant. □

Theorem E.2 is not a peculiarity of Cobweb: synergistic information—information about Y held by a set of variables jointly and by none of them alone [25]—is invisible to any module that sees one of them. Back-propagation escapes it because each module receives the error of the composition. A Cobweb tree receives no error, so the decision has to be taken where the information is composed.

E.2 Verification at the point of composition

Theorem E.4 (Task selection by verification)**.** *Let $|Y| \le B$ and let $\mathcal{A}$ be a finite set of task assignments. For each $A \in \mathcal{A}$ let $\hat{F}_{A,N}$ be the output of the forest that Cobweb learns with assignment A from a training stream of length N, with $|\hat{F}_{A,N}| \le B$ and $\mathbb{E}\int |\hat{F}_{A,N} - F_A|^2 d\mu \to 0$ (Theorems C.2, D.5 and D.10 give this for activation forests, label forests and single trees; for a single tree, and for a label forest under the conditions of Theorem D.10, $F_A = f$). Let $\hat{A}_N$ minimise the mean squared error of Y on an independent validation stream of length $N_v(N) \to \infty$. Then*

$$\Pr\Big[\Delta(\hat{A}_N) = \min_{A \in \mathcal{A}} \Delta(A)\Big] \to 1.$$

In particular, if $\mathcal{A}$ contains a sufficient assignment, the selected assignment is sufficient with probability tending to one and the selected forest is consistent.

Proof. Let $\sigma^2 = \mathbb{E}\operatorname{Var}(Y \mid X)$. Conditionally on the training stream, the expected validation error of A is $R_N(A) = \sigma^2 + \mathbb{E}[(f - \hat{F}_{A,N})^2 \mid \text{train}]$, and $\mathbb{E}[(f - \hat{F}_{A,N})^2 \mid \text{train}] \to \Delta(A)$ in probability, because $\|f - \hat{F}_{A,N}\|_{L_2(\mu)} - \|f - F_A\|_{L_2(\mu)}$ is bounded by $\|\hat{F}_{A,N} - F_A\|_{L_2(\mu)} \to 0$. The validation average of the bounded squared errors differs from $R_N(A)$ by $O_P(N_v^{-1/2})$ (Chebyshev). With $\gamma = \min\{\Delta(A) - \Delta^* : \Delta(A) > \Delta^*\} > 0$, $\Delta^* = \min_A \Delta(A)$, every assignment with $\Delta(A) > \Delta^*$ has a larger validation error than every minimiser with probability tending to one, and $\mathcal{A}$ is finite. □

The validation error at the output is the outcome of the composing tree. With the log loss in place of the squared error the same rule would estimate the conditional entropy $H(Y \mid U_o)$ and select the task assignment that maximises the predictive information $I(Y; U_o)$; proving this needs the convergence of the trees' predictive distributions, not only of their means, which we leave open. Category utility measures the same kind of quantity—the information a partition carries about an attribute—when Cobweb chooses among its operators; task selection evaluates it one level up, for the outcome and the composing tree's input. In the online protocol the prequential error $\frac{1}{N}\sum_i (Y_i - \hat{F}_{A,i-1}(X_i))^2$ plays the role of the validation error [5]; it converges to the same limit whenever the online forest's risk tends to that of F_A, by a Cesàro argument.

E.3 Task operators

Definition E.5 (Task operators and synergy deficit)**.** Let a tree with outcome V compose two subtrees σ_1, σ_2 with inputs C_1, C_2. The *task operators* at this composition are

- ACT: σ_j gets the outcome V and passes its activation;
- LABEL: σ_j gets no outcome and passes its label;
- HYBRID: σ_j gets the outcome V and passes its activation and label;
- CROSS: σ_j gets a coordinate of C_{3-j} as outcome and passes its activation;
- MERGE: σ_1, σ_2 are replaced by one tree with input $C_1 \cup C_2$.

The *synergy deficit* of the pair is $\Delta_{\text{ACT}}(\sigma_1, \sigma_2) = \mathbb{E}\,\mathrm{Var}(V \mid a_1, a_2) - \mathbb{E}\,\mathrm{Var}(V \mid C_1, C_2) \geq 0$, with $a_j = \mathbb{E}[V \mid C_j]$; it is zero exactly when ACT loses nothing at this composition, and it is estimated by the difference of the validation errors of ACT and MERGE. It is zero whenever both activations are injective functions of their inputs—then (a_1, a_2) determines (C_1, C_2)—whatever the interaction, so it detects the interactions that activations lose, not all interactions.

MERGE and its inverse mirror Cobweb's own merge and split one level up: they decide whether two inputs are perceived by one concept hierarchy or by two.

Procedure E.6 (Task induction by verification)**.**

1. Start with one tree per raw coordinate (or per given block).
2. Estimate the synergy deficit of every pair; merge the pairs whose deficit is significantly positive (union–find over these pairs).
3. Compose the resulting groups bottom-up with two-input trees; at each composition choose the operator in $\{\text{ACT}, \text{LABEL}, \text{HYBRID}, \text{CROSS}\}$ by the validation error of the composing tree's outcome.

By Theorem E.4 each choice in step 3 is consistent given the subtrees below it, among the candidates that satisfy its convergence hypothesis (activation forests under Theorem D.5, label forests under Theorem D.10; hybrid and cross-view compositions are not covered); the procedure as a whole is greedy and carries no global guarantee. By Theorem E.2 no step can be replaced by a rule that looks at a subtree alone. Pairwise deficits are also blind to synergy of higher order: for the parity of three coordinates every pair is independent of Y (Corollary E.3), every pairwise deficit is zero and step 2 merges nothing. The candidates of step 3 must therefore always include the merged tree or a label forest, on which the selection can fall back. Every tree still learns only with `ifit`: the procedure chooses *what* a tree predicts—which observable it will see—never how it learns.

F Experiments in full

This appendix gives the setup and all results summarised in Section 5. The experiments probe the statements of Appendices B to E and repeat the KAN paper's battery [14]. Most runs use the implementation's defaults, which lie outside the hypotheses of the convergence theorems (online protocol, PATH mixture, a leaf budget, the matching tolerance; Remarks C.3 and C.4); where a run is outside a statement's hypotheses we say so, and read it as a test of the statement's prediction rather than as an instance of it. The constructions in the proofs are checked exactly by the verifier (Appendix G). The experiments also do not claim that Cobweb forests are competitive regressors: a Cobweb tree is a nonparametric smoother, and on most smooth low-dimensional targets the gradient-trained MLP and KAN are far more accurate.

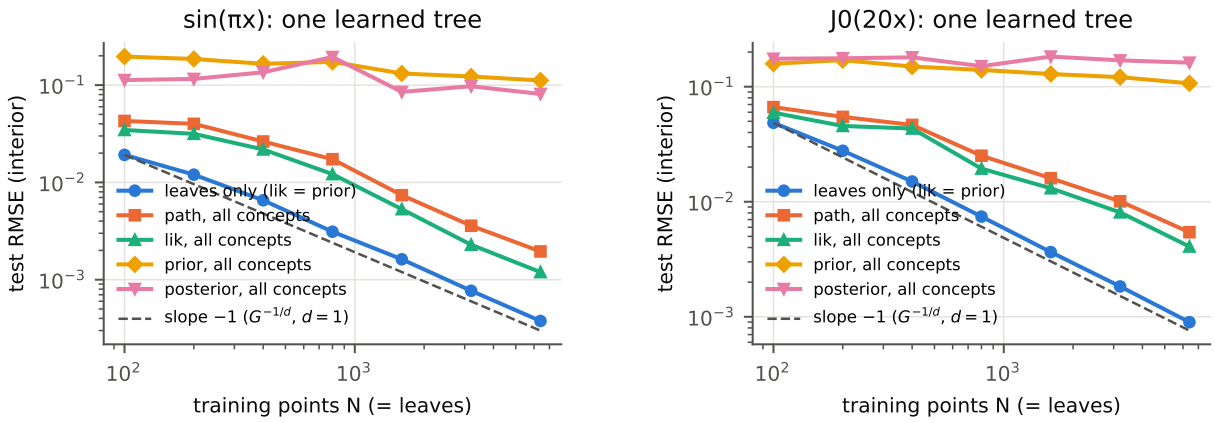


Figure 5: One learned tree, variance floor at the data spacing, full expansion. The leaves-only mixture falls with slope -1; the PATH and LIK mixtures over all concepts fall more slowly; the PRIOR mixture over all concepts falls only like $1/\log N$ and the POSTERIOR mixture stays high (Remark B.4). Numbers in Table 4.

Setup. Trees are the hybrid Cobweb of `cobweb-private` with the additions listed in the repository (masked queries, node identifiers, leaf-counted budgets); a test checks that it reproduces the original trees and predictions exactly. Unless stated otherwise every tree predicts with the leaves-only PATH mixture and a budget of 300 leaves, every forest is trained with the online protocol (Definition A.6), continuous attributes are measured in units of their training standard deviation, and the variance floor is $h^2 = (\kappa N^{-1/(d+4)})^2$ for a tree with d continuous inputs. κ and the label depth (from $\{2, 3, 4, 6\}$) are chosen on a fresh validation set, except in the task experiments of Appendix F.3, which fix $\kappa = 0.25$ so that candidates differ only in their tasks. MLP and KAN (B-spline edges as in Liu et al. [14]) are trained with Adam and early stopping, with width or grid chosen on the same validation set. Toy targets are noise-free on $[-1, 1]^n$ and are scored on 2000 fresh points; errors are RMSE divided by the standard deviation of f (of Y where f is constant, as in the sign world P_b), averaged over three seeds, and over five for the locality, selection and multi-view experiments. Every result file records the hashes of the code that produced it, and the verifier checks that every reported result comes from a build without the budget bug described in the repository (Appendix G).

F.1 One tree

Figure 5 and Table 4 show the four weightings on learned trees. These runs are outside the hypotheses of Theorem B.2 and Proposition B.3, which concern a constructed grid tree, and of Theorem C.2: the variance floor equals the data spacing ($h \asymp 1/N$), the interpolation regime of a noise-free target, not the regime $Nh^d \to \infty$. They show the same pattern as the constructions: for $\sin(\pi x)$ the leaves-only error falls from 0.019 at $N = 100$ to 0.00038 at $N = 6400$, a factor of about four per quadrupling of N, while the PRIOR mixture over all concepts stays between 0.11 and 0.20 and the POSTERIOR mixture between 0.08 and 0.14 at the tabulated N (and 0.19 at $N = 800$ in the figure) (between 0.15 and 0.19 for $J_0(20x)$).

target	mixture	$N = 100$	$N = 400$	$N = 1600$	$N = 6400$
$\sin(\pi x)$	leaves only	0.019	0.0066	0.0016	0.00038
$\sin(\pi x)$	path, all	0.043	0.026	0.0074	0.002
$\sin(\pi x)$	lik, all	0.035	0.022	0.0053	0.0012
$\sin(\pi x)$	prior, all	0.197	0.166	0.132	0.112
$\sin(\pi x)$	posterior, all	0.113	0.135	0.085	0.081
$J_0(20x)$	leaves only	0.049	0.015	0.0037	0.0009
$J_0(20x)$	path, all	0.066	0.047	0.016	0.0054
$J_0(20x)$	lik, all	0.059	0.043	0.013	0.0041
$J_0(20x)$	prior, all	0.158	0.150	0.129	0.107
$J_0(20x)$	posterior, all	0.175	0.180	0.183	0.162

Table 4: Interior test RMSE of one learned tree (mean of three seeds), the data of Fig. 5.

N	leaf budget	κ	level 0 $\lvert S\rvert\leq 1$	level 1 $\lvert S\rvert\leq 2$	level 2 $\lvert S\rvert\leq 4$	level 3 $\lvert S\rvert\leq 8$	level 4 $\lvert S\rvert\leq 16$	level 5 $\lvert S\rvert\leq 32$	level 6 $\lvert S\rvert\leq 64$	output RMSE/sd
4000	100	0.25	1.38	1.42	1.40	1.27	1.12	1.08	1.02	1.00
4000	100	1.0	3.16	1.46	1.41	1.38	1.19	1.08	1.06	1.01
4000	1000	1.0	0.57	0.87	1.06	1.10	1.06	1.06	1.02	1.00
4000	4000	1.0	0.57	0.82	0.99	1.02	1.01	1.00	1.00	1.00
16000	1000	1.0	0.90	0.92	1.16	1.07	1.04	0.98	0.97	0.98
16000	4000	1.0	0.56	0.63	0.86	0.91	0.90	0.89	0.89	0.92

Table 5: The 100-D funnel forest level by level (N training points, one seed): RMSE of every tree's activation divided by the standard deviation of its ideal message m_S, averaged over the trees of a level. A ratio near 1 means the activation carries no information about m_S.

F.2 The KAN battery

Figure 2 and Table 2 show five things. (i) One tree's error falls with N on every target of dimension at most four, the behaviour Theorem C.2 predicts (the runs use the PATH mixture with a budget, outside its hypotheses). On $\sin(\pi x)$ and $\exp(\sin \pi x + y^2)$ at $N = 16000$ it is 7 to 11 times behind the MLP and 16 to 34 times behind the KAN; on $J_0(20x)$ it is ahead of both at $N = 250$ and ahead of the KAN, whose grid limits it, at $N = 16000$. (ii) On xy the funnel forest stays between 0.99 and 1.02 at every N—what Proposition D.8 predicts in the limit, under hypotheses (split protocol, count-weighted full expansion) that these online, budget-limited runs do not meet— while labels (0.22 at $N = 16000$) and hybrid messages (0.19) learn; the single tree over both coordinates is better still (0.011), because with $n = 2$ it has the same fan-in. (iii) On the 4-D target the funnel forest over the pairs $(x_1, x_2), (x_3, x_4)$, which is sufficient by Proposition D.7, beats the single tree at small N (0.46 against 0.57 at $N = 1000$) and loses at $N = 16000$ (0.33 against 0.28). Here $n = k + 2$, so the bounds of Remark D.13 have equal exponents. With a noisy target the remark's bounds favour the forest, which leads at small N (0.63 against 0.81 at $N = 250$) but only ties at $N = 16000$ (0.320 against 0.319). In both cases the measured slopes run the other way—about -0.16 for the forest against -0.22 (noisy) and -0.24 (noise-free) for the single tree over $N = 250$ to 16000—so at these sample sizes the comparison is not governed by the bounds' exponents; the forest's composition error, which the bounds ignore, is a likely reason that we have not isolated. (iv) On the 100-D target no Cobweb design gets below 0.95 (the single tree is worse than the constant predictor, at 1.55 to 1.61), while the KAN reaches 0.002 and the MLP 0.40.

(v) Table 5 locates the 100-D failure. Each univariate tree must estimate $\mathbb{E}[Y \mid X_i]$, whose

		tree perceiving x_2 alone		composing tree (test RMSE)			
N	world	val. RMSE	act. sd	funnel	labels (d=3)	hybrid	merged
1000	P_a	1.004	0.062	1.008	0.675	0.642	0.270
1000	P_b	1.002	0.080	1.006	1.143	1.116	1.080
4000	P_a	1.000	0.047	1.064	0.460	0.510	0.244
4000	P_b	1.002	0.048	1.028	1.074	1.056	1.018
16000	P_a	1.003	0.069	1.164	0.429	0.471	0.215
16000	P_b	1.004	0.073	1.092	1.051	1.034	1.007

Table 6: The two sign worlds of Theorem E.2 (five seeds). Left: a tree that perceives x_2 and predicts Y—its validation RMSE equals that of the constant predictor (1.00) and its activation is flat in both worlds. Right: the composing tree's test RMSE for four task assignments.

standard deviation is about a tenth of that of Y. At $N = 4000$, with a budget of 100 leaves the estimate is dominated by noise (ratio 1.4 for $\kappa = 0.25$, 3.2 for $\kappa = 1$); with 1000 or 4000 leaves it carries signal (ratio 0.57), but the trees one level up read these noisy activations, and the signal is gone by the third level. At $N = 16000$ with 4000 leaves part of it survives to the top (ratios 0.56 to 0.91, output 0.92): with this budget the composition improves with N, in the direction Theorem D.5 predicts for the limit, but with 1000 leaves the low levels get worse with N (0.57 $\to$ 0.90 at level 0), so the improvement depends on the budget; these runs are in any case outside the theorem (online protocol, a budget, and messages with critical points, outside (A2)). This is the finite-sample face of the condition behind (A4): the children's errors are not small compared with what their parents must resolve, and errors compound through the levels. A KAN does not face this problem, because its gradient corrects all inner functions against the error of the output.

F.3 Cognitive tasks

Locality. Table 6 reproduces Theorem E.2 in data: the tree that perceives x_2 has the same validation error and the same flat activation in both worlds, so nothing it can compute tells it whether x_2 matters. The composing tree separates the worlds: under P_a the merged tree reaches 0.22 and the label forest 0.43 at $N = 16000$, while under P_b, where $f \equiv 0$ and the constant predictor is optimal, every assignment stays near 1. The funnel forest fails under P_a, where its error even rises above the constant predictor's (1.16 at $N = 16000$), because its online, budget-limited activations are noise that the output tree fits.

Selection. Selecting by the validation error at the output (Table 7) never selects the funnel forest on xy or on the sign target. On the 4-D target it selects a forest over the pairs in 14 of 15 runs (in one run at $N = 500$ it selects the univariate funnel forest), over the merged 4-D tree. The selected forest's test error is within one percent of the best candidate's in every cell. On two-dimensional targets the merged tree wins, as it should: it has the same fan-in as the forest and no composition error. These runs test the selection rule rather than instantiate Theorem E.4: the theorem assumes that every candidate converges, whereas here every candidate is trained online with the PATH mixture and a budget, and the fixed-depth label and hybrid candidates are not covered by our consistency results.

Synergy. Figure 6 estimates the synergy deficit of every pair of coordinates, the quantity step 2 of Procedure E.6 thresholds. On the 4-D target the within-block deficits are 0.18 and 0.23 of $\mathrm{Var}(Y)$

target	N	selected (count of 5)	funnel	selected	best candidate
xy	500	merge (5)	1.020	0.071	0.071
xy	2000	merge (5)	1.000	0.032	0.032
xy	8000	merge (5)	1.003	0.019	0.019
sign	500	merge (5)	0.996	0.309	0.309
sign	2000	merge (5)	1.007	0.245	0.245
sign	8000	merge (5)	1.075	0.231	0.231
expsin	500	merge (5)	0.232	0.115	0.115
expsin	2000	merge (5)	0.145	0.055	0.055
expsin	8000	merge (5)	0.111	0.034	0.034
nested4	500	funnel-pairs (4), funnel (1)	0.773	0.662	0.657
nested4	2000	funnel-pairs (5)	0.651	0.451	0.450
nested4	8000	funnel-pairs (3), hybrid-pairs (2)	0.622	0.334	0.334

Table 7: Task selection by verification (Theorem E.4; five seeds). Candidates: the funnel forest, label forests of depth 2, 3 and 4, a hybrid forest, one merged tree and, for the 4-D target, funnel and hybrid forests over the pairs. Columns: how often each candidate was selected, and the test RMSE/sd of the funnel forest, of the selected forest and of the best candidate in hindsight.

and the cross-block deficits are within 0.04 of zero, so the threshold recovers the blocks $\{x_1, x_2\}$ and $\{x_3, x_4\}$. On housing the deficit of longitude with latitude is large in every run (0.13 to 0.17 at $N = 4000$): location is an interaction that activations do not carry. Rooms with bedrooms averages 0.13 in Fig. 6 but varies between data splits (-0.01 to 0.25), so the induction runs of Table 8, which estimate their own deficits, mostly leave it below the threshold. At $N = 16000$ every housing pair shows a positive deficit (between 0.03 and 0.22), so the threshold of step 2 must account for sample size. The deficit vanishes whenever both activations are injective functions of their inputs, whatever the interaction (Definition E.5), so a positive deficit marks an interaction that the activations lose, not every interaction.

Task induction. Table 8 runs Procedure E.6 end to end. The groups it forms are the interactions that activations lose: in $q(E_f + Bv \sin\theta)$ it merges B, v and θ and keeps E_f apart, and it also keeps q apart, although q multiplies every term, because q's activation is injective; in $I.6.20b$ it merges θ with θ_1, whose difference the formula uses; in $I.29.16$ it merges the two angles; on the 4-D target it recovers the two blocks. The selected forest equals the best candidate, or is within one percent of it, in every row but one; it beats the single tree where the induced structure is real (the 4-D target and housing at both sizes, e.g. 0.495 against 0.598 on housing at $N = 16000$, where the threshold $\tau = 0.05$ merges every input but the selected forests, at $\tau = 0.1$ or 0.2, merge only longitude with latitude in all three runs; and narrowly on $I.12.11$ and $I.50.26$); and it falls back on the single tree on the other formulas. The exception is $II.6.15a$, where the selected forest (0.546) is worse than the single tree (0.519), which was among the candidates. The generic balanced forest is the worst design on every formula: a forest needs the right graph, and verification at the composing tree finds one.

Cross-view tasks. In the two-view model (Fig. 7) the cross-view forest, whose inner trees never see Y, keeps improving when the target depends on the shared latent (0.45 at $N = 500$, 0.27 at $N = 32000$), as Proposition D.9 predicts, while the funnel forest stalls near 0.39: here the funnel messages $\mathbb{E}[\sin 2Z \mid X^{(v)}]$ are not invertible functions of the per-view posterior means, so the funnel assignment is not sufficient. When the target is specific to one view the cross-view forest stalls

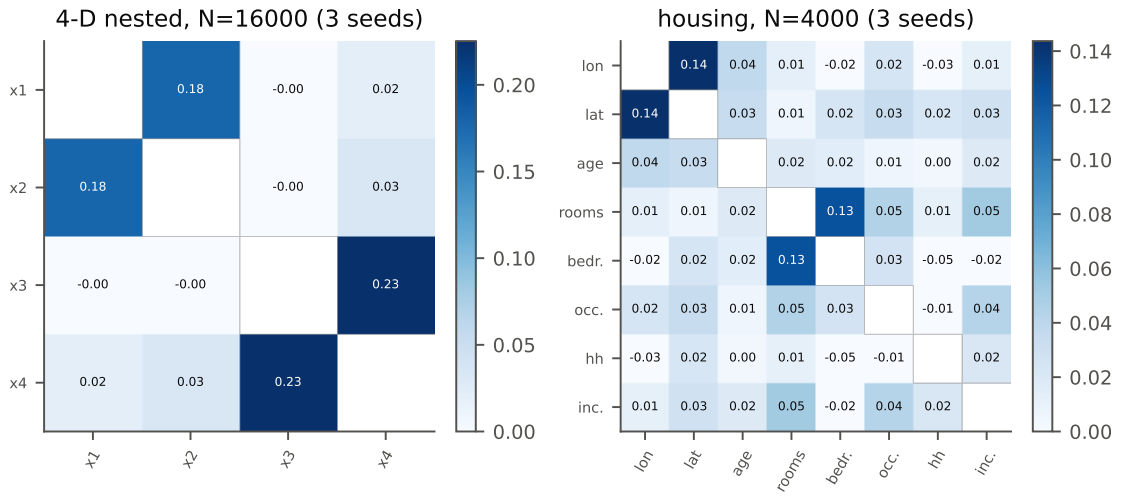


Figure 6: Synergy deficits (Definition E.5): validation MSE of the funnel pair forest minus that of one tree over the pair, divided by $\mathrm{Var}(Y)$. Left: the 4-D target; the two blocks stand out and the cross-block deficits are near zero. Right: California housing; the largest deficits are longitude–latitude and rooms–bedrooms.

near 0.75: a positive deficit, as predicted after Proposition D.9 (the remark predicts its sign, not its size). The single tree over all four coordinates is best in both panels at these sample sizes. (The Gaussian views are unbounded, so Theorem D.5 itself does not cover these runs.)

F.4 Continual learning, interpretability and labels

No forgetting. Lemma C.1 says that nothing a tree has learned is deleted. On the KAN paper's continual task (Fig. 3 and Table 9) the tree's error on everything seen so far stays between 0.037 and 0.048 through all five phases; the MLP forgets (its error reaches 10.8), and the KAN with a fine grid forgets partly (0.17).

Interpretability. A KAN's edges can be plotted, and so can a Cobweb forest's trees. On $\exp(\sin \pi x_1 + x_2^2)$ the univariate trees learn $c_1 e^{\sin \pi x_1}$ and $c_2 e^{x_2^2}$ (Fig. 4), the inner functions of the target up to the monotone maps H_S of Proposition D.7, without gradients.

Label cells. Theorem D.10 needs the cells of the labels to shrink. In one dimension the cells of a fixed depth shrink slowly as the tree grows; in two dimensions they do not (Fig. 8). Labels become fine enough only if their depth grows with N; the depths our experiments try are fixed (2 to 6), so the label forests of this section are outside Theorem D.10. We also tried the cognitively attractive choice of the *basic level*—the concept of largest expected pointwise mutual information on the branch, which Cobweb computes: for continuous attributes it lies near the leaves (mean depth 10 to 11), because a concept's information grows as its variance shrinks towards the variance floor, and its labels did worse than any fixed depth on xy.

F.5 Real data

Feynman equations. With a balanced graph over the inputs in their listed order—a graph that ignores the structure of each formula—the forests lose to the single tree on every equation except

target	N	induced groups ($\tau = 0.05$)	one tree	balanced	selected	best	seeds
I.12.11	4000	$\{q\}\{E_f\}\{B, v, \theta\}$	0.216	0.594	0.212	0.212	3
I.13.12	4000	$\{m_1\}\{m_2\}\{r_1\}\{r_2\}\{G\}$	0.311	0.634	0.311	0.311	3
I.29.16	4000	$\{x_1\}\{x_2\}\{\theta_1, \theta_2\}$	0.217	0.853	0.217	0.217	3
I.37.4	4000	$\{I_1\}\{I_2\}\{\delta\}$	0.106	0.299	0.106	0.106	3
I.50.26	4000	$\{x_1\}\{\omega, t\}\{\alpha\}$	0.345	0.826	0.327	0.326	3
I.6.20	4000	$\{\sigma, \theta\}$	0.023	0.350	0.023	0.023	3
I.6.20b	4000	$\{\sigma\}\{\theta, \theta_1\}$	0.083	0.536	0.083	0.083	3
I.9.18	4000	$\{m_1\}\{m_2\}\{x_2\}\{x_1\}\{y_2\}\{y_1\}\{z_2\}\{z_1\}\{G\}$	0.438	0.602	0.438	0.438	3
II.6.15a	4000	$\{\epsilon, r\}\{p_d\}\{x, y\}\{z\}$	0.519	0.562	0.546	0.494	3
housing	4000	$\{lon, lat\}\{age\}\{rooms\}\{bedr\}\{occ\}\{hh\}\{inc\}$	0.636	0.633	0.547	0.547	3
housing	16000	$\{lon, lat, age, rooms, bedr, occ, hh, inc\}$	0.598	0.617	0.495	0.495	3
nested4	4000	$\{x_1, x_2\}\{x_3, x_4\}$	0.450	0.607	0.407	0.407	3
nested4	16000	$\{x_1, x_2\}\{x_3, x_4\}$	0.300	0.626	0.294	0.294	3

Table 8: Task induction end to end (Procedure E.6; three seeds, $\kappa = 0.25$ for every candidate, so the single-tree errors differ from those of Tables 2 and 10, which tune κ). Groups formed by step 2 at threshold $\tau = 0.05$ (most frequent over seeds, each run estimating its own deficits) and test RMSE/sd of the single tree, of the generic balanced forest over the inputs, of the forest selected on validation data among the single tree, the balanced forest and the induced forests for $\tau \in \{0.02, 0.05, 0.1, 0.2\}$ with activation or hybrid messages, and of the best candidate in hindsight.

method	RMSE on all data seen so far, after phase 1..5				
cobweb	0.037	0.048	0.045	0.045	0.045
mlp	0.014	0.965	10.756	7.506	8.241
kan	0.009	0.107	0.161	0.160	0.168

Table 9: Continual learning: RMSE on all data seen so far after each phase (three seeds).

$II.6.15a$, where the hybrid forest is better (0.421 against 0.486; Table 10). The MLP and the KAN are better than every Cobweb design on every equation, by factors from about two (the single tree against the KAN on $I.29.16$) to more than a hundred. Table 8 shows that Procedure E.6 finds structure in the formulas and matches or beats the single tree on every equation but $II.6.15a$; it does not close the gap to the gradient-trained models.

Housing. The funnel forest over the natural pairs (longitude–latitude, age–rooms, bedrooms–occupancy, households–income) beats the single tree at every N (0.276 against 0.303 at $N = 16000$; Table 11), and the univariate graph does not; the pairs graph keeps together longitude and latitude, the strongest interaction of Fig. 6. MLP and KAN are better still.

MNIST. Patch trees that predict the class and pass their predicted class distributions beat the single tree over all pixels at $N = 1000$ and 5000 and match it at 20000 (94.4 ± 0.5 against 94.2 ± 0.3), and the hybrid forest reaches 94.9% at $N = 20000$, level with the KAN (94.5%) and below the MLP (96.8%; Table 3). Categorizing patch trees (the Convolutional-Cobweb protocol) are weaker than outcome-predicting ones at every N (70.7% to 86.2%): categorization alone passes less information about the class than outcome prediction does. Patch trees that predict the centre pixels of their neighbouring patches—perceptual prediction, which never sees the class—are the weakest design (37% to 52% on average). A plausible reading, which we have not tested, is that the class is not what neighbouring centre pixels share—the analogue of the multi-view redundancy that Proposition D.9

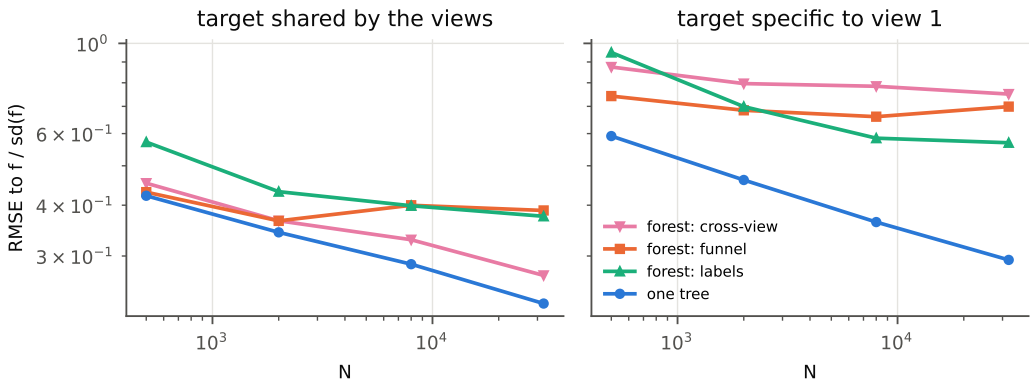


Figure 7: The two-view Gaussian model of Proposition D.9 (five seeds): RMSE to the optimal predictor f, divided by $\mathrm{sd}(f)$. Left: $Y = \sin(2Z)$ depends on the shared latent. Right: Y is the first noise coordinate of view 1.

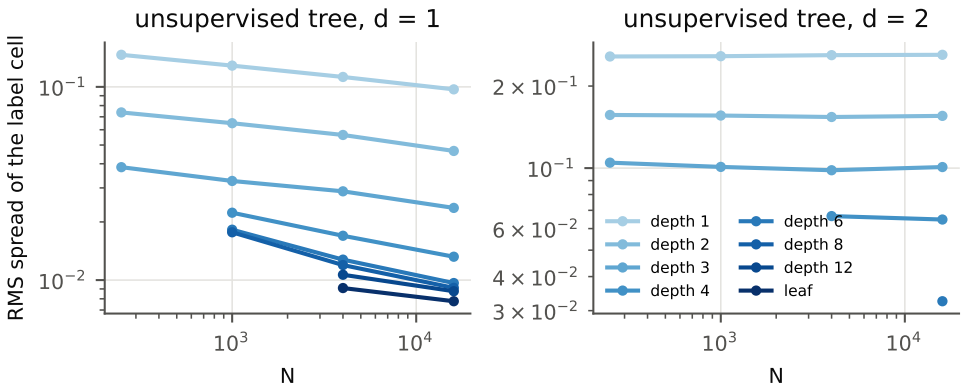


Figure 8: Spread of the cell of a label at fixed depth, for unsupervised trees over uniform data.

assumes in its Gaussian setting—so the activations discard most of what identifies the digit.

Text. On next-word prediction (Table 12) the trees' most probable word is the unknown-word token, so their accuracy equals its frequency, and their cross-entropy is worse than that of an interpolated n-gram; at $N = 20000$ the funnel label forest improves the single tree's cross-entropy slightly (5.77 against 5.88; the unsupervised one does not, 5.87), and neither improves its accuracy. A Cobweb language model needs the architecture of a dedicated study; we report these numbers as the limit of the generic forest.

G Verification

Executable verifier. `verification/checks.py` instantiates the objects each statement talks about with the real C++ tree—learned by `ifit`, or built exactly for the constructions in the proofs (depth-one grid trees, the label-forest construction, interwoven trees)—and tests the statement's conclusion; `verification/run.py` writes the table below. The first check compares Definition A.3 and Definition A.1, implemented from the definitions, with the code's scores and mixtures.

equation	n	single	act	label	hybrid	mlp	kan
I.12.11	5	0.140	0.592	0.578	0.479	0.004	**0.002**
I.13.12	5	0.281	0.626	0.916	0.492	0.006	**0.005**
I.29.16	4	0.182	0.854	0.880	0.422	**0.011**	0.095
I.37.4	3	0.092	0.257	0.360	0.206	**0.003**	0.003
I.50.26	4	0.308	0.801	0.871	0.793	**0.016**	0.125
I.6.20	2	0.021	0.339	0.097	0.094	0.001	**0.001**
I.6.20a	1	0.002	–	–	–	0.000	**0.000**
I.6.20b	3	0.078	0.511	0.605	0.541	**0.003**	0.020
I.9.18	9	0.322	0.476	0.891	0.573	**0.003**	0.015
II.6.15a	6	0.486	0.556	0.878	0.421	**0.013**	0.024

Table 10: Feynman equations, $N = 4000$: test RMSE/sd(f), mean of three seeds; forests use a balanced graph over the inputs in their listed order.

N	single	act:pairs	act:uni	label:pairs	label:uni	hybrid:uni	hybrid:pairs	mlp	kan
1000	0.362±0.009	0.336±0.011	0.366±0.002	0.464±0.027	0.535±0.009	0.408±0.016	0.357±0.011	0.308±0.009	0.286±0.008
4000	0.325±0.006	0.299±0.005	0.344±0.005	0.461±0.005	0.496±0.035	0.339±0.012	0.314±0.008	0.271±0.005	0.262±0.004
16000	0.303±0.002	0.276±0.003	0.346±0.006	0.432±0.008	0.496±0.014	0.314±0.005	0.279±0.009	0.250±0.001	0.249±0.002

Table 11: California housing: test RMSE of the log median value (three seeds).

result	statement	check
pass	Definition A.3	Def. multi-node prediction and Def. concept equal the implementation
pass	Lemma C.1	Leaf preservation (no leaf deleted; additivity; leaves hold identical copies)
pass	Lemma B.1	Concept-bias certificate holds for every weighting and budget
pass	Theorem B.2, Proposition B.3	Grid tree: error and root weight vanish (lik, path, leaves); prior plateaus
pass	Theorem B.9	Label-forest construction meets its error bound; labels are nearest grid points
pass	Proposition B.11	Interwoven tree = convex combination of the subtrees (lik, path)
pass	Lemma B.5, Theorem B.6	Composed constructed trees approximate $\Phi(\varphi_1 + \varphi_2)$
pass	Theorem C.2	Learned tree, leaves-only lik = Nadaraya–Watson (path is not)
pass	Proposition D.8	xy: funnel activations vanish, funnel forest fails; label error falls with N at every depth
pass	Theorem E.2	Sign construction: local laws equal; composition separates the worlds
pass	Proposition D.9	Posterior given all views = function of per-view posterior means
pass	Remark C.4	Branch-and-bound stop returns a sub-optimal concept (continuous and nominal-only trees)
pass	Proposition D.7	H_S strictly increasing; funnel forest on a monotone-additive target converges
pass	Corollary E.3	3-way parity: pairs independent of Y; label forest learns it, funnel does not
pass	Theorem D.5	Split protocol: sup error to the ideal forest falls with N
pass	Theorem E.4	Validation at the output never selects the funnel forest on

N	ngram	mlp	single	label-unsup	label-funnel
20000	17.8 / 5.07	20.9 / 5.10	20.9 / 5.88	20.9 / 5.87	20.9 / 5.77
100000	19.1 / 4.84	21.7 / 4.66	20.9 / 5.14	–	–
300000	19.9 / 4.68	23.6 / 4.41	20.9 / 5.04	–	–

Table 12: Next word on OpenWebText from the previous four words (2000-word vocabulary): top-1 accuracy (%) / cross-entropy (nats). An accuracy of 20.9 is the frequency of the unknown-word token.

Formal proofs. The discrete cores of the statements below are proved in Lean 4 with Mathlib [6, 23] (`lean/`, built by `lake build`). The check `check_lean` rebuilds the project, prints the axioms of every listed theorem and fails if any depends on `sorryAx` or if a source file contains `sorry`, `admit` or a new `axiom`; every theorem uses only Lean's standard axioms (`propext`, `Classical.choice`, `Quot.sound`).

statement	Lean theorem	scope
Proposition D.8, Corollary E.3 (constant messages)	`validity_nonneg`, `condExp_const_of_validity_zero`	a tree whose input carries no information about its outcome emits a constant message (any finite distribution)
Theorem E.2(i)–(ii)	`sign_worlds_local_law`, `local_rule_same`, `sign_worlds_differ`	the sign skeleton on $\{\pm 1\}^3$
Corollary E.3	`parity_fibre_balanced`	parity of all n bits ($J = [n]$, signs as bits), every S missing a bit
Proposition B.11	`merged_prediction`, `merged_weights`	the mixture identity, m subtrees
Proposition B.7	`Chain.fanin_one`	every fan-in-one forest
Theorem B.6 (count)	`ka_tree_count`	$4n^2 + 4n$ trees, $n \geq 1$

Analytic statements (rates, consistency) are not formalised; for them the executable checks and the review below carry the verification.

Adversarial review. Each statement is also checked by an independent reviewer—a fresh language-model agent for every round—instructed to break it, to settle questions by computation, and to read nothing but this document, its tables and the two C++ files it formalises (`verification/REVIEW_PROMPT.md`). The reviewer was calibrated before it was trusted: on a copy of an earlier draft with eight planted errors (three of them real mistakes of that draft) it found all eight with correct diagnoses, and every other flag it raised was a genuine error, which we fixed. The same prompt was then run on successive drafts:

round	outcome
calibration	8/8 planted errors found; extra findings (label consistency with atoms, exact matching, crossover exponents) fixed
2	no statement false under its hypotheses; experiments text contradicting tables, overstated remarks and summaries; fixed
3	no statement false under its hypotheses; prior/posterior weightings on hierarchies (verified by check C19), branch-and-bound repair, minor gaps; fixed
4	no statement invalid; four minor gaps (an unsatisfiable form of a hypothesis of Theorem D.10, an unused density hypothesis, a cited uniform law, an ambiguous phrase) and five text items; fixed

The reports, the drafts they reviewed and the reviewers' own scripts are in `verification/reviews/`; how each finding was resolved is in `verification/REVIEW.md`.

References

[1] Vladimir I. Arnold. On functions of three variables. *Doklady Akademii Nauk SSSR*, 114: 679–681, 1957.

[2] Avrim Blum and Tom Mitchell. Combining labeled and unlabeled data with co-training. In *Proceedings of the Eleventh Annual Conference on Computational Learning Theory*, pages 92–100, 1998.

[3] Andy Clark. Whatever next? Predictive brains, situated agents, and the future of cognitive science. *Behavioral and Brain Sciences*, 36(3):181–204, 2013.

[4] James E. Corter and Mark A. Gluck. Explaining basic categories: Feature predictability and information. *Psychological Bulletin*, 111(2):291–303, 1992.

[5] A. Philip Dawid. Present position and potential developments: Some personal views. Statistical theory: The prequential approach. *Journal of the Royal Statistical Society, Series A*, 147(2): 278–292, 1984.

[6] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[7] Luc Devroye. The uniform convergence of the Nadaraya–Watson regression function estimate. *Canadian Journal of Statistics*, 6(2):179–191, 1978.

[8] Luc Devroye and Adam Krzyżak. An equivalence theorem for L_1 convergence of the kernel regression estimate. *Journal of Statistical Planning and Inference*, 23:71–82, 1989.

[9] Uwe Einmahl and David M. Mason. Uniform in bandwidth consistency of kernel-type function estimators. *The Annals of Statistics*, 33(3):1380–1403, 2005.

[10] Douglas H. Fisher. Knowledge acquisition via incremental conceptual clustering. *Machine Learning*, 2(2):139–172, 1987.

[11] Robert L. Goldstone. Perceptual learning. *Annual Review of Psychology*, 49:585–612, 1998.

[12] László Györfi, Michael Kohler, Adam Krzyżak, and Harro Walk. *A Distribution-Free Theory of Nonparametric Regression*. Springer, 2002.

[13] Andrey N. Kolmogorov. On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition. *Doklady Akademii Nauk SSSR*, 114:953–956, 1957.

[14] Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. KAN: Kolmogorov–Arnold networks. In *International Conference on Learning Representations*, 2025. arXiv:2404.19756.

[15] George G. Lorentz. *Approximation of Functions*. Holt, Rinehart and Winston, 1966.

[16] Y. P. Mack and Bernard W. Silverman. Weak and strong uniform consistency of kernel regression estimates. *Zeitschrift für Wahrscheinlichkeitstheorie und verwandte Gebiete*, 61:405–415, 1982.

[17] Christopher J. MacLellan and Harshil Thakur. Convolutional Cobweb: A model of incremental learning from 2D images. In *Advances in Cognitive Systems*, 2021. arXiv:2201.06740.

[18] Christopher J. MacLellan, Karthik Singaravadivelan, Xin Lian, Zekun Wang, and Pat Langley. A rational account of categorization based on information theory. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, 2026. arXiv:2603.29895.

[19] Marvin Minsky. Steps toward artificial intelligence. *Proceedings of the IRE*, 49(1):8–30, 1961.

[20] Robert A. Rescorla and Allan R. Wagner. A theory of Pavlovian conditioning: Variations in the effectiveness of reinforcement and nonreinforcement. In Abraham H. Black and William F. Prokasy, editors, *Classical Conditioning II: Current Research and Theory*, pages 64–99. Appleton-Century-Crofts, 1972.

[21] Karthik Singaravadivelan and Pat Langley. A unified framework of concepts and chunks. Manuscript, Advances in Cognitive Systems, 2026.

[22] David A. Sprecher. On the structure of continuous functions of several variables. *Transactions of the American Mathematical Society*, 115:340–355, 1965.

[23] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020.

[24] Christopher Tosh, Akshay Krishnamurthy, and Daniel Hsu. Contrastive learning, multi-view redundancy, and linear models. In *Proceedings of the 32nd International Conference on Algorithmic Learning Theory*, volume 132 of *PMLR*, pages 1179–1206, 2021.

[25] Paul L. Williams and Randall D. Beer. Nonnegative decomposition of multivariate information, 2010. arXiv:1004.2515.